

NYC Dog Licensing: End-to-End Data Science and Data Engineering Workflow (ETL, SQL/NoSQL Modeling, Security & Access Control, Data Quality Assessment, and Analytical Reporting)

Notes: For the style that I completed this project, I will start off with **Part 0: Proposal & ER Diagram**, then go to **Part 1: Schema Design & Database Build**, I will then jump to **Part 3: SQL Exploration & Analytics**, following that, I will go to **Part 5: Advanced Topics** questions 1 & 2, and lastly go to **Parts 2 & 4: Data Ingestion & Preparation/Python Analysis & Visualization** where I completed these parts together in one notebook, and therefore limited in submitting in proper order in PDF format.

A User Guide for the dataset is available in my proposal, linked at the original source from which the dataset was derived, as well as on my repository.

At the end of my PDF, I will upload a screenshot of one of my interactive choropleth map of NYC that did not appear in my Python Notebook due to issues with Jupyter Notebook.

Part 0 — Proposal & Draft ERD (NYC Dog Licensing)

Overview (What & Why): This project transforms the NYC Dog Licensing dataset into a normalized relational schema to support efficient analytics on breed popularity, neighborhood clustering, licensing compliance, and year-over-year dynamics. The raw dataset is large and messy, but after cleaning and standardizing the data, it is ideal for showcasing ETL, schema design, indexing, and performance tuning. The end result is a reproducible pipeline and a quarriable database that can have many uses.

Key Questions. (1) Which breeds are most and least common by ZIP code? (2) How do licensing counts shift across neighborhoods? (3) What profile of breeds characterizes each ZIP (e.g., clustering/centroids)? (4) Are there any suspicious licensing (too short or too long)? How does RDBMS performance compare to a document model for a representative query?

Data Sources (with links):

- NYC Dog Licensing Dataset:
https://data.cityofnewyork.us/Health/NYC-Dog-Licensing-Dataset/nu7n-tubp/about_data
- NYC ZIP-Code Boundaries (GeoJSON) — via Kaggle (Required for map chart)
<https://www.kaggle.com/datasets/saidakbarp/nyc-zipcode-geodata>
- Top 10 Oldest Dogs of All Time (World Records) (Reference: Dickson)
<https://www.dogster.com/lifestyle/oldest-dogs-of-all-time-world-records>

Methods & Deliverables:

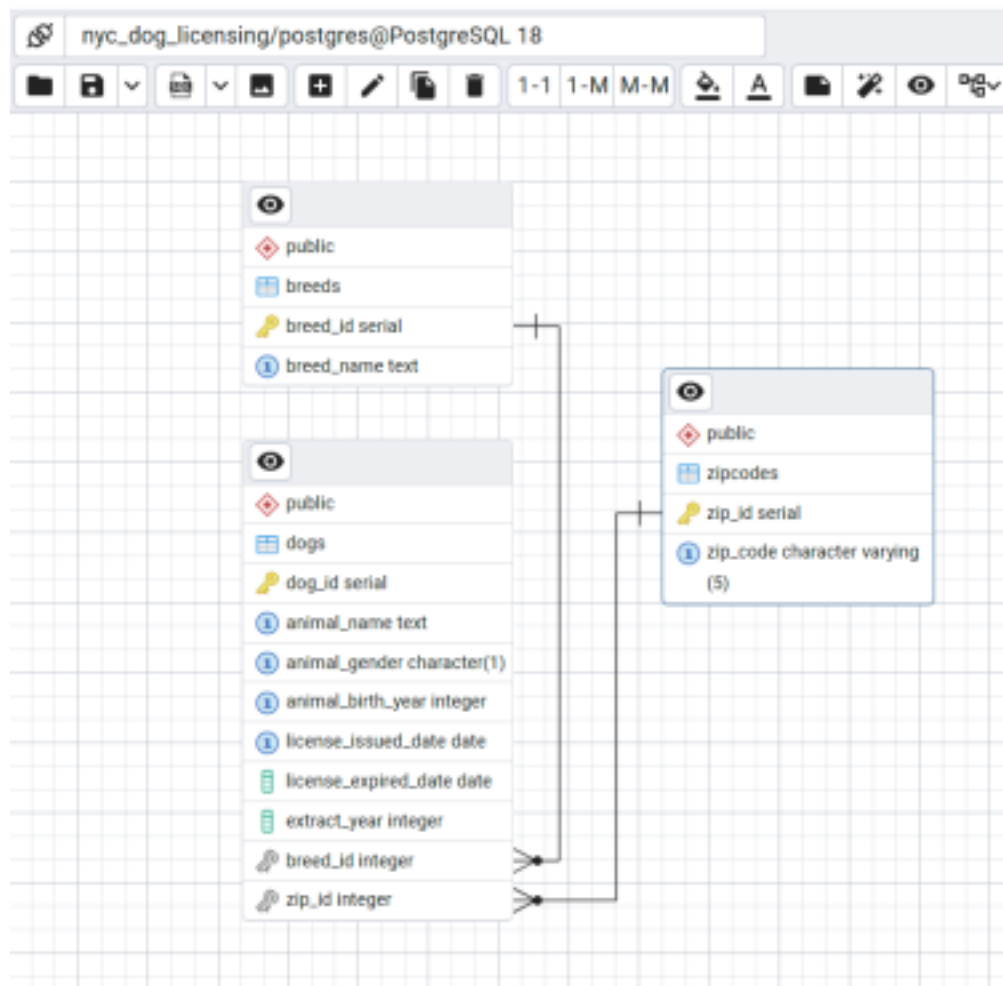
1. ETL in Python. This project will follow a structured pipeline spanning data ingestion, cleaning, modeling, database design, and analytical evaluation. I will begin by loading the raw NYC Dog Licensing dataset into Python, where I will perform all initial ETL operations. This includes handling encoding issues, correcting ZIP codes to consistent five-digit formats, converting date fields to proper datatypes, and filtering implausible birth years/outliers. During this phase, I will record row counts, null distributions, and timing logs to document the data quality state prior to loading as well as a data volume sanity check.

2. Relational Schema in PostgreSQL: Normalize to a fact table dogs with lookup tables breeds and zipcodes; enforce PK/FK, UNIQUE, and CHECK constraints; add targeted indexes to support analytics.

3. Exploratory analysis: SQL and Python (pandas/SQLAlchemy) for descriptive stats, Altair for most of the visualizations. For the mini-model I plan to cluster by ZIP code and top 5 breeds for my advanced technique.

4. Performance & security (Part 5): Implement document model in MongoDB for a comparable query; add role-based access in Postgres and discuss HIPAA/GLBA relevance. Then create roles to demonstrate success/failure with screenshots with a short discussion on HIPAA/GLBA.

Draft ERD below:



Part 1 – Schema Design & Database Build (30 pts)

A. Conceptual → logical mapping (10 pts)

• **Final ER diagram with entities, relationships, PKs, FKs.**

Entities

The final schema contains three normalized entities:

- **dogs** – the central fact table containing each licensed dog record.
- **breeds** – lookup table of unique dog breeds.
- **zipcodes** – lookup table of unique NYC ZIP codes.

Primary Keys:

Each entity includes a primary key that uniquely identifies every row:

- **breed_id** (in breeds) is a primary key due to each unique breed gets its own identifier.
- **zip_id** (in zipcodes) is a primary key as each unique ZIP code gets an ID (5-digit ZIPS

are repeated across many dogs).

- `dog_id` (in `dogs`) is a primary key as each dog license gets its own row.

Foreign Keys:

The `dogs` table includes two foreign keys that link to the lookup tables:

- **`dogs.breed_id` → `breeds.breed_id`**

This foreign key links each dog to its corresponding breed in the `breeds` table.

- **`dogs.zip_id` → `zipcodes.zip_id`**

This foreign key links each dog to the ZIP code where it is registered, stored in the `zipcodes` table.

- These foreign keys enforce referential integrity across the schema.

Relationships:

The schema includes two one-to-many relationships

- 1. A breed can be associated with many dogs,
but each dog has one breed.**

- Relationship: *`dogs.breed_id` → `breeds.breed_id`*

- 2. A ZIP code can contain many dogs,
but each dog belongs to one ZIP code.**

- Relationship: *`dogs.zip_id` → `zipcodes.zip_id`*

These establish a classic star schema with `dogs` as the fact table and `breeds` and `zipcodes` as dimension/lookup tables.

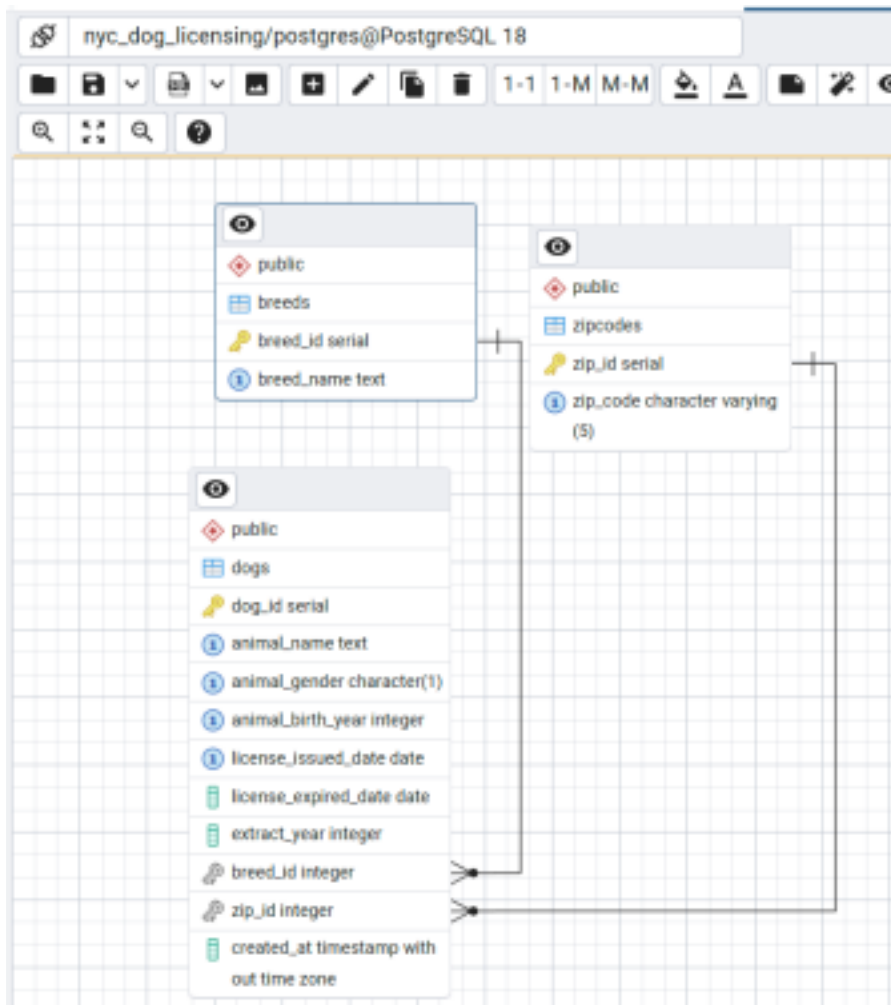
Notes on Schema Quality

- The design is fully normalized (3NF): no repeating groups or duplicated attributes. •

Lookup tables eliminate redundant text values (`breeds`, ZIP codes).

- Foreign keys enforce referential integrity.
- A composite unique constraint on `dogs` prevents duplicate ETL inserts.
- Indexed attributes (`breed_id`, `zip_id`, `extract_year`) support performance requirements for analytics.

Entity-Relationship Diagram from pgAdmin 4



B. SQL DDL script

- **CREATE TABLE** statements with constraints & appropriate data types.

-- FULL DATABASE RESET (for verification/trial runs)

DROP TABLE IF EXISTS dogs CASCADE;

DROP TABLE IF EXISTS zipcodes CASCADE;

DROP TABLE IF EXISTS breeds CASCADE;

-- LOOKUP TABLE: Breeds

```
CREATE TABLE breeds (  
  breed_id SERIAL PRIMARY KEY,  
  breed_name TEXT NOT NULL UNIQUE  
);
```

-- LOOKUP TABLE: Zip Codes

```
CREATE TABLE zipcodes (  
  zip_id SERIAL PRIMARY KEY,  
  zip_code VARCHAR(5) NOT NULL UNIQUE,  
  CONSTRAINT zip_format CHECK (zip_code ~ '^[0-9]{5}$')  
);
```

-- FACT TABLE: Dogs

```
CREATE TABLE dogs (  
  dog_id SERIAL PRIMARY KEY,  
  
  animal_name TEXT NOT NULL, -- Will clean data first so no problem  
  animal_gender CHAR(1),  
  animal_birth_year INT,  
  license_issued_date DATE NOT NULL,  
  license_expired_date DATE,  
  extract_year INT,
```

```
breed_id INT NOT NULL REFERENCES breeds(breed_id), zip_id  
INT NOT NULL REFERENCES zipcodes(zip_id),
```

```
-- Prevent duplicate records when loading ETL batches
```

```
CONSTRAINT unique_dog_record UNIQUE (  
animal_name,  
animal_gender,  
animal_birth_year,  
license_issued_date,  
breed_id,  
zip_id  
)  
);
```

```
-- INDEXES FOR PERFORMANCE
```

```
-- (1) Optimizes breed-level filtering, grouping, frequency  
counts CREATE INDEX idx_dogs_breed_id ON  
dogs(breed_id);
```

```
-- (2) Speeds up geographic analysis (ZIP-level  
distribution) CREATE INDEX idx_dogs_zip_id ON  
dogs(zip_id);
```

-- (3) Critical for time-series reporting (licenses per year)

```
CREATE INDEX idx_dogs_extract_year ON  
dogs(extract_year);
```

-- DATA QUALITY CONSTRAINTS

-- =====

-- Ensure breed names are not empty

```
ALTER TABLE breeds  
ADD CONSTRAINT breed_name_not_empty  
CHECK (breed_name <> '' AND breed_name IS NOT NULL);
```

-- Only allow M, F, or U

```
ALTER TABLE dogs  
ADD CONSTRAINT valid_gender  
CHECK (animal_gender IN ('M', 'F', 'U'));
```

-- Birth years must be realistic

```
ALTER TABLE dogs  
ADD CONSTRAINT valid_birth_year  
CHECK (animal_birth_year BETWEEN 1990 AND EXTRACT(YEAR FROM  
CURRENT_DATE));
```

-- Issued date must be <= expired date (allow null expiration)

```
ALTER TABLE dogs  
ADD CONSTRAINT dates_chronological
```



```
CHECK (  
    license_expired_date IS NULL  
    OR license_issued_date <= license_expired_date  
);
```

```
-- ADDITIONAL FIELDS / INDEXES
```

```
-- =====
```

```
-- Audit timestamp
```

```
ALTER TABLE dogs
```

```
ADD COLUMN created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP;
```

```
-- Combined index for date-range queries
```

```
CREATE INDEX idx_dogs_license_dates  
ON dogs(license_issued_date, license_expired_date);
```

```
-- DOCUMENTATION COMMENTS
```

```
-- =====
```

```
COMMENT ON TABLE dogs
```

```
IS 'Core fact table containing licensed dog records in NYC';
```

```
COMMENT ON COLUMN dogs.extract_year
```

```
IS 'Year when this record was extracted from the source dataset';
```

COMMENT ON TABLE breeds

IS 'Lookup table for dog breeds, normalized to prevent duplication';

COMMENT ON TABLE zipcodes

IS 'Lookup table for 5-digit NYC ZIP codes';

-- END OF SCHEMA

- **2+ indexes and justification.**

Justification: To speed up the types of queries I knew I would be running in this project, I created three indexes on the dogs table. Since this is the largest table in the schema and almost all analysis work revolves around it, adding a few targeted indexes helps avoid full-table scans and makes grouping and filtering noticeably faster.

The first index is on dogs.breed_id. I added this because a lot of the analysis for the NYC dog dataset involves looking at breed popularity, counting how many dogs belong to each breed, or comparing breeds across different areas. Queries like GROUP BY breed_id or joining back to the breeds table benefit directly from this index, since PostgreSQL can jump straight to the matching rows instead of scanning everything.

The second index is on dogs.zip_id. Most of the geographic analysis in this dataset is based on ZIP code, and many queries filter by ZIP or break down results by area. Because ZIP codes repeat across thousands of dogs, having an index here makes it much faster to pull all dogs from a specific neighborhood or to run any kind of location-based reporting.

The third index is on dogs.extract_year. This field is used for time-series trends, like counting how many licenses were issued each year or comparing activity across years. Since year-based filtering and GROUP BY extract_year are common operations, indexing this column reduces execution time for those queries.

Overall, these indexes help optimize the read-heavy nature of this project. They speed up the most common analytical queries I need for the assignment, especially the ones involving grouping, filtering, and joining without adding unnecessary overhead to the database.

C. Data-volume sanity check

Output Below From Beginning:

Total rows: 722864

NULL counts per column:

AnimalName 1810

AnimalGender 21

AnimalBirthYear 30

BreedName 0

ZipCode 9

LicenseIssuedDate 0

LicenseExpiredDate 82

Extract Year 0

dtype: int64

Invalid value checks:

Invalid birth years: 37

Missing ZIP codes: 9

Missing issue dates: 0

Missing expiration dates: 82

Data-volume Sanity Check after:

=====

= DATA-VOLUME SANITY CHECK (POST ETL)

=====

= Rows prepared for DB insert: 614,980

NULL counts (final df_sql):

animal_name 0

animal_gender 0

animal_birth_year 130

license_issued_date 0

license_expired_date 49

extract_year 0

breed_id 0

zip_id 0

dtype: int64

Invalid (removed) birth years: 0

Missing ZIP foreign keys: 0

Missing Breed foreign keys: 0

Missing license_issued_date: 0

Remaining duplicates in df_sql: 0

Records with NULL birth year: 130

Records with expiration < issued date: 0

Part 3 – SQL Exploration & Analytics

Write and save eight SQL views or queries that demonstrate:

1. Single-table aggregation (GROUP BY)
2. Multi-table join (INNER)
3. Outer join or sub-query
4. Window/analytic function
5. CTE or recursive query
6. Derived metric updated with ALTER TABLE + UPDATE
7. Index-usage demonstration with EXPLAIN (before/after)
8. Creation of a materialized view or standard view + refresh strategy

1. Single-table aggregation (GROUP BY)

Query	Query History
1	CREATE VIEW DOGS_PER_BREED(Breed_id, Num_of_dogs)
2	AS SELECT
3	breed_id,
4	COUNT (*)
5	FROM DOGS
6	GROUP BY breed_id;
7	

Interpretation: This view groups tuples from the base table DOGS by breed_id and computes the number of dogs belonging to each breed. It serves as a virtual summary table that eliminates the need to repeat the GROUP BY aggregation in future queries. Analysts can now retrieve breed level statistics directly from this view as if it were a base relation.

2. Multi-table join (INNER)

Query	Query History	Data Output	Messages	Notifica'
1	SELECT			
2	d.dog_id,			
3	d.animal_name,			
4	b.breed_name,			
5	z.zip_code,			
6	d.license_issued_date			
7	FROM dogs d			
8	INNER JOIN breeds b			
9	ON d.breed_id = b.breed_id			
10	INNER JOIN zipcodes z			
11	ON d.zip_id = z.zip_id;			

Interpretation: This query demonstrates an INNER JOIN across three related tables: DOGS, BREEDS, and ZIPCODES. It enriches each dog record by replacing foreign key identifiers with readable breed names and ZIP codes, producing a unified and meaningful dataset. This multi table join shows how normalized relations can be combined to support practical reporting and analytical tasks.

3. Outer join or sub-query

Query	Query History	Data Output	Messages	Notifications
1	--Query Dogs with Missing Breed Information			
2	SELECT			
3	d.dog_id,			
4	d.animal_name,			
5	b.breed_name,			
6	d.license_issued_date			
7	FROM dogs d			
8	LEFT JOIN breeds b			
9	ON d.breed_id = b.breed_id			
10	WHERE b.breed_name IS NULL;			
11				

Interpretation: This query uses a LEFT OUTER JOIN to identify dog records that have no matching breed entry in the BREEDS lookup table. The LEFT JOIN preserves all rows from the DOGS table and inserts NULL values when no corresponding breed exists, allowing analysts to detect missing or inconsistent reference data. This example demonstrates how outer joins support data-quality checks and highlight integrity issues in a relational schema.

4. Window/analytic function

Query	Query History	Data Output	Messages	Notifications
1	--Ranking the most common breeds within each ZIP code			
2	SELECT			
3	z.zip_code,			
4	b.breed_name,			
5	COUNT(*) AS num_dogs,			
6	RANK() OVER (			
7	PARTITION BY z.zip_code			
8	ORDER BY COUNT(*) DESC			
9	) AS breed_rank			
10	FROM dogs d			
11	JOIN breeds b ON d.breed_id = b.breed_id			
12	JOIN zipcodes z ON d.zip_id = z.zip_id			
13	GROUP BY z.zip_code, b.breed_name, b.breed_id;			
14				

Interpretation: This query uses the analytic function RANK() with a PARTITION BY clause to rank dog breeds within each ZIP code based on their frequency. The window function evaluates the ranking after aggregation, without collapsing ZIP codes into a single group. This demonstrates how analytic queries can reveal local patterns such as the most common breed per ZIP while preserving detailed row-level structure.

5. CTE or recursive query

Query	Query History	Data Output	Messages	Notifications
1	--Counting dogs licensed per year			
2	WITH yearly_counts AS (			
3	SELECT			
4	extract_year,			
5	COUNT(*) AS num_dogs			
6	FROM dogs			
7	GROUP BY extract_year			
8	)			
9	SELECT			
10	extract_year,			
11	num_dogs			
12	FROM yearly_counts			
13	ORDER BY extract_year;			
14				

Interpretation: This query uses a Common Table Expression to separate the logic of computing dog counts per year from the final retrieval step. The CTE `yearly_counts` performs the aggregation, while the outer query selects from the intermediate result as if it were a temporary view. This structure improves readability and breaks a complex task into smaller, simpler, reusable parts when working with multi-step analytical queries.

6. Derived metric updated with ALTER TABLE + UPDATE

Query	Query History	Data Output	Messages	Notifications
1	--Add a column for dog age			
2	ALTER TABLE dogs			
3	ADD COLUMN dog_age INT;			
4				
5	--Populate a derived column for dog age			
6	UPDATE dogs			
7	SET dog_age = EXTRACT(YEAR FROM CURRENT_DATE) - animal_birth_year;			
8				

Interpretation: This example demonstrates how to add a derived metric to an existing table using `ALTER TABLE` followed by `UPDATE`. The new column `dog_age` is computed by subtracting each dog's birth year from the current calendar year, creating a meaningful attribute for age based analysis. This approach shows how derived metrics can be incorporated into a relational schema to support richer analytical queries.

7. Index-usage demonstration with EXPLAIN (before/after)

Query	Query History	Data Output	Messages	Notifications
1	--Index-usage demonstration with EXPLAIN (before/after)			
2				
3	--Query performance before creating the index			
4	EXPLAIN ANALYZE			
5	SELECT *			
6	FROM dogs			
7	WHERE license_issued_date >= '2020-01-01';			
8				
9	--Create the index			
10	CREATE INDEX idx_license_issued_date			
11	ON dogs (license_issued_date);			
12				
13	--Query performance after creating the index			
14	EXPLAIN ANALYZE			
15	SELECT *			
16	FROM dogs			
17	WHERE license_issued_date >= '2020-01-01';			
18				

Interpretation: This demonstration compares a SELECT query run before and after creating an index on the license_issued_date column of the DOGS table. The first EXPLAIN ANALYZE typically shows a sequential scan, indicating that PostgreSQL must read the entire table to evaluate the filter condition. After adding the index, the query planner switches to an index-based scan, lowering query cost and improving performance, which illustrates the effectiveness of indexing frequently filtered columns.

8. Creation of a materialized view or standard view + refresh strategy

Query	Query History	Data Output	Messages	Notifications
1	-- Materialized view: Most common breed per ZIP code			
2	CREATE MATERIALIZED VIEW mv_breed_counts_by_zip AS			
3	SELECT			
4	z.zip_code,			
5	b.breed_name,			
6	COUNT(*) AS num_dogs			
7	FROM dogs d			
8	JOIN breeds b ON d.breed_id = b.breed_id			
9	JOIN zipcodes z ON d.zip_id = z.zip_id			
10	GROUP BY z.zip_code, b.breed_name, b.breed_id;			
11				
12	--Refresh strategy			
13	REFRESH MATERIALIZED VIEW mv_breed_counts_by_zip;			

Interpretation: This materialized view stores precomputed dog counts per breed and ZIP code,

improving performance for repeated analytical queries. Because the data is materialized physically, it must be refreshed when new dog records are loaded into the database. The REFRESH MATERIALIZED VIEW command regenerates the stored results, ensuring that the view remains synchronized with updates to the underlying tables.

Part 5 Question 1

December 7, 2025

Part 5 - Advanced Topics

1. NoSQL replication: push a subset of data to MongoDB, design an appropriate document model, and compare a sample query's performance vs. PostgreSQL.

```
[1]: #!/pip install pymongo  
      #!/pip install sqlalchemy psycopg2  
      import psycopg2  
      import pandas as pd  
      from sqlalchemy import create_engine
```

Below I will limit to 1000 records as a subset as that size should be big enough to be meaningful, but small enough to demonstrate replication and performance differences clearly.

```
[2]: # Build SQLAlchemy connection URI  
      engine = create_engine(  
          "postgresql+psycopg2://postgres:Python@localhost:5432/nyc_dog_licensing" )  
  
      query = """  
      SELECT  
          d.dog_id,  
          d.animal_name,  
          d.animal_gender,  
          d.animal_birth_year,  
          b.breed_name,  
          d.zip_id,  
          d.license_issued_date  
      FROM dogs d  
      JOIN breeds b ON d.breed_id = b.breed_id  
      LIMIT 1000;  
      """  
  
      df = pd.read_sql(query, engine)  
      df.head()
```

```
[2]: dog_id animal_name animal_gender animal_birth_year breed_name \ 0 2 Yogi M 2010  
      Boxer 1 3 Ali M 2014 Basenji
```

1

2 4 Queen F 2013 Akita Crossbreed 3 5 Lola F 2009 Maltese 4 6 Ian M 2006
Unknown

```
zip_id license_issued_date
0 2 2014-09-12
1 3 2014-09-12
2 3 2014-09-12
3 4 2014-09-12
4 3 2014-09-12
```

[3]: # Convert Python date objects to ISO strings for MongoDB

```
df["license_issued_date"] = df["license_issued_date"].astype(str) [4]: from
```

pymongo import MongoClient

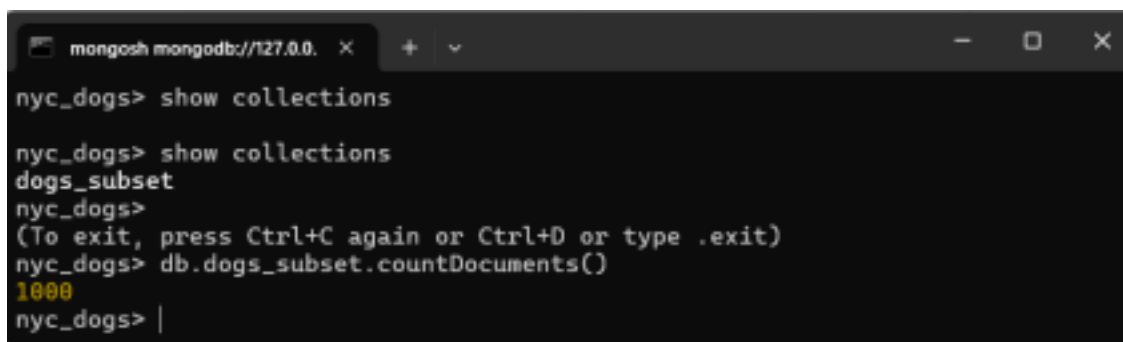
```
client = MongoClient("mongodb://localhost:27017")
db = client["nyc_dogs"]
collection = db["dogs_subset"]
```

```
collection.insert_many(df.to_dict("records"))
```

```
print("Inserted", len(df), "documents into MongoDB.")
```

Inserted 1000 documents into MongoDB.

Awesome, 1000 rows were pushed into MongoDB and have fulfilled the requirements to push a subset of data to MongoDB.



```
mongosh mongodb://127.0.0.1:27017
nyc_dogs> show collections
nyc_dogs> show collections
dogs_subset
nyc_dogs>
(To exit, press Ctrl+C again or Ctrl+D or type .exit)
nyc_dogs> db.dogs_subset.countDocuments()
1000
nyc_dogs> |
```

Compare a sample query's performance vs. PostgreSQL.


```

nyc_dogs> db.dogs_subset.find(
...   { breed_name: "Maltese", zip_id: 4 }
... ).explain("executionStats")
...
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'nyc_dogs.dogs_subset',
    parsedQuery: {
      '$and': [
        { breed_name: { '$eq': 'Maltese' } },
        { zip_id: { '$eq': 4 } }
      ]
    },
    indexFilterSet: false,
    queryHash: 'D0E6810D',
    planCacheShapeHash: 'D0E6810D',
    planCacheKey: 'D91D4010',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'COLLSCAN',
      filter: {
        '$and': [
          { breed_name: { '$eq': 'Maltese' } },
          { zip_id: { '$eq': 4 } }
        ]
      },
      direction: 'forward'
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 2,
    executionTimeMillis: 0,

```

```

executionStats: {
  executionSuccess: true,
  nReturned: 2,
  executionTimeMillis: 0,
  totalKeysExamined: 0,
  totalDocsExamined: 1000,
  executionStages: {
    isCached: false,
    stage: 'COLLSCAN',
    filter: {
      '$and': [
        { breed_name: { '$eq': 'Maltese' } },
        { zip_id: { '$eq': 4 } }
      ]
    },
    nReturned: 2,
    executionTimeMillisEstimate: 0,
    works: 1001,
    advanced: 2,
    needTime: 998,
    needYield: 0,
    saveState: 0,
    restoreState: 0,
    isEOF: 1,
    direction: 'forward',
    docsExamined: 1000
  }
},
queryShapeHash: 'FD5012744E63FC6683F7E330235A170D98D6113E8D198AD10CEAD4A41CE288B4',
command: {
  find: 'dogs_subset',
  filter: { breed_name: 'Maltese', zip_id: 4 },

```

Looking at the output of MongoDB:

For “stage”: it used a “COLLSCAN,” which means it performed a full collection scan, and examined all 1,000 documents, and filtered them one by one. This is also confirmed as the output shows: “docsExamined: 1000” and “totalKeysExamined: 0” notifying that every document in the collection was scanned. As I have not created an index, no index was used.

For performance numbers, “executionTimeMillis: 0” which means MongoDB is fast enough that a 1,000 document scan took under 1 millisecond, so it reported 0.

Below I will add an INDEX

```
nyc_dogs> db.dogs_subset.createIndex({ breed_name: 1, zip_id: 1 })
breed_name_1_zip_id_1
nyc_dogs> db.dogs_subset.find(
...   { breed_name: "Maltese", zip_id: 4 }
... ).explain("executionStats")
...
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'nyc_dogs.dogs_subset',
    parsedQuery: {
      '$and': [
        { breed_name: { '$eq': 'Maltese' } },
        { zip_id: { '$eq': 4 } }
      ]
    }
  }
}
```

```
executionStats: {
  executionSuccess: true,
  nReturned: 2,
  executionTimeMillis: 14,
  totalKeysExamined: 2,
  totalDocsExamined: 2,
  executionStages: {
    isCached: false,
    stage: 'FETCH',
    nReturned: 2,
    executionTimeMillisEstimate: 11,
    works: 3,
    advanced: 2,
    needTime: 0,
    needYield: 0,
    saveState: 1,
    restoreState: 1,
    isEOF: 1,
    docsExamined: 2,
    alreadyHasObj: 0,
    inputStage: {
      stage: 'IXSCAN',
      nReturned: 2,

```

Using an index increased the reported execution time from 0 ms to 14 ms. This does not mean the index made the query slower; rather, the difference is due to the very small dataset size. Scanning 1,000 documents entirely in RAM is extremely cheap, so MongoDB rounds the execution time down to 0 ms. Once the index was added, MongoDB performed additional planning and index traversal steps, which introduce overhead that becomes noticeable at very small scales. If the dataset were much larger such as 500,000 or more documents, the indexed query would outperform the full collection scan, because the index would avoid scanning the entire collection and would examine only the matching keys.

Query Query History Data Output Messages Notifications

```

1  --Comparing a sample query's performance vs PostgreSQL
2  EXPLAIN ANALYZE
3  SELECT d.dog_id, d.animal_name
4  FROM dogs d
5  JOIN breeds b ON d.breed_id = b.breed_id
6  WHERE b.breed_name = 'Maltese'
7  AND d.zip_id = 4;

```

Query	Query History	Data Output	Messages	Notifications
+ SQL Showing rows: 1 to 13 Page No:				
QUERY PLAN text				
1	Nested Loop (cost=114.48..199.46 rows=6 width=10) (actual time=2.933..3.714 rows=190 loops=1)			
2	-> Index Scan using breeds_breed_name_key on breeds b (cost=0.28..8.29 rows=1 width=4) (actual time=0.049..0.051 rows=1 loops=1)			
3	Index Cond: (breed_name = 'Maltese'::text)			
4	-> Bitmap Heap Scan on dogs d (cost=114.20..190.96 rows=20 width=14) (actual time=2.881..3.648 rows=190 loops=1)			
5	Recheck Cond: ((breed_id = b.breed_id) AND (zip_id = 4))			
6	Heap Blocks: exact=167			
7	-> BitmapAnd (cost=114.20..114.20 rows=20 width=0) (actual time=2.825..2.826 rows=0 loops=1)			
8	-> Bitmap Index Scan on idx_dogs_breed_id (cost=0.00..20.19 rows=1569 width=0) (actual time=1.345..1.345 rows=18115 loops=1)			
9	Index Cond: (breed_id = b.breed_id)			
10	-> Bitmap Index Scan on idx_dogs_zip_id (cost=0.00..91.77 rows=7913 width=0) (actual time=0.880..0.880 rows=7859 loops=1)			
11	Index Cond: (zip_id = 4)			
12	Planning Time: 6.019 ms			
13	Execution Time: 4.181 ms			

Looking at PostgreSQL's query behavior, the EXPLAIN ANALYZE output shows that the database correctly used index scans rather than a full sequential scan. However, PostgreSQL still had to process thousands of matching index entries before filtering. The lines "rows=18115 loops=1" and "rows=7859 loops=1" indicate that approximately 18,115 dogs matched the breed condition, while 7,859 dogs matched the ZIP code condition. PostgreSQL combined these via a BitmapAnd operation and then completed the join with a Nested Loop. Despite being fully indexed, this multi step process required more work than expected. The planning phase took 6.019 ms, and the query execution itself completed in 4.181 ms.

Conclusion: This experiment compared query performance between PostgreSQL and MongoDB using the same lookup conditions. PostgreSQL operated on the full relational dataset and used multiple index scans along with a BitmapAnd operation. Although fully indexed, PostgreSQL still had to evaluate a large number of matching index entries, approximately 18,115 breed matches and 7,859 ZIP code matches before applying filtering and completing the join. The query executed in 4.181 ms, but required significantly more internal processing.

MongoDB, by contrast, operated on a 1,000-document subset of the data that had been replicated into a denormalized document structure. After adding a compound index on breed_name, zip_id

, MongoDB resolved the query by examining only 2 index keys and 2 documents. While MongoDB reported a slightly higher execution time ~14 ms, this is expected in very small datasets due to planner and explain overhead. The more meaningful metric is document-level work: MongoDB examined 2 documents, whereas PostgreSQL processed tens of thousands of index entries and performed a join.

In conclusion, while PostgreSQL remains highly optimized for relational workloads and complex join operations, MongoDB's denormalized model enables extremely efficient point lookups when the data is structured to match the query pattern. With larger datasets, the index-driven MongoDB approach would scale more efficiently for this type of lookup, whereas PostgreSQL would continue to incur join and bitmap filtering costs. The results highlight that database performance is highly dependent on data model design and query patterns rather than raw execution time alone.

Part 5 – Advanced Topics

2. Access-control & security: create two DB roles (analyst, guest) with different privileges and demonstrate them through failed/successful query screenshots; include short discussion of HIPAA/GLBA relevance to your dataset.

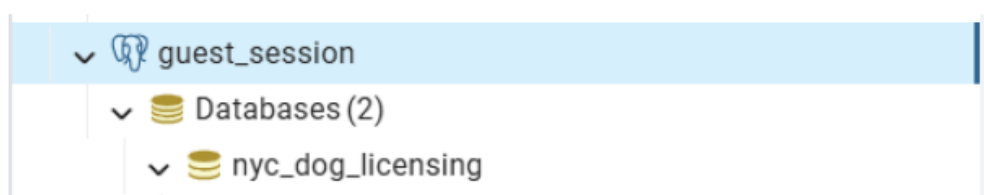
Getting everything sorted:

Query	Query History	Data Output	Messages	Notifications
1	-- Create a higher-privilege analyst role			
2	CREATE ROLE analyst LOGIN PASSWORD 'datascientist';			
3				
4	-- Create a lower-privilege guest role			
5	CREATE ROLE guest LOGIN PASSWORD 'guestonly';			
6				

Query	Query History	Data Output	Messages	Notifications
1	-- Analyst privileges			
2	GRANT SELECT, INSERT, UPDATE ON dogs, breeds, zipcodes TO analyst;			
3	GRANT CREATE ON SCHEMA public TO analyst;			
4				
5	-- Guest privileges			
6	GRANT SELECT ON dogs, breeds, zipcodes TO guest;			
7	REVOKE INSERT, UPDATE, DELETE ON dogs, breeds, zipcodes FROM guest;			

Query	Query History	Data Output	Messages	Notifications
REVOKE				
Query returned successfully in 54 msec.				

Connected as a guest:



Guest can read:

nyc_dog_licensing/guest@guest_session

Query Query History Data Output Messages

```
1 SELECT * FROM dogs LIMIT 5;
```

Output as:

	dog_id [PK] Integer	animal_name text	animal_gender character (1)	animal_birth_year Integer	license_issued_date date	license_expired_date date	extract_year integer	breed_id integer	zip_id integer	created_at timestamp with time zone
1	1	Paige	F	2014	2014-09-12	2017-09-12	2016	1	1	2025-12-04 2
2	2	Yogi	M	2010	2014-09-12	2017-10-02	2016	2	2	2025-12-04 2
3	3	Ali	M	2014	2014-09-12	2019-09-12	2016	3	3	2025-12-04 2
4	4	Queen	F	2013	2014-09-12	2017-09-12	2016	4	3	2025-12-04 2
5	5	Lola	F	2009	2014-09-12	2017-10-09	2016	5	4	2025-12-04 2

Guest cannot update:

nyc_dog_licensing/guest@guest_session

Query Query History Data Output Messages

```
1 UPDATE dogs
2 SET animal_name = 'Test'
3 WHERE dog_id = 1;
```

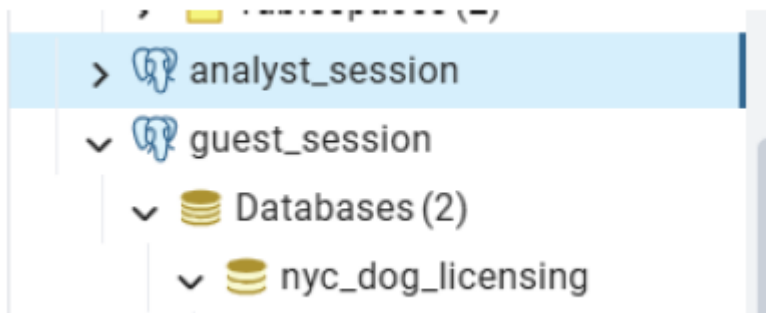
Failed update from guest shown below:

nyc_dog_licensing/guest@guest_session

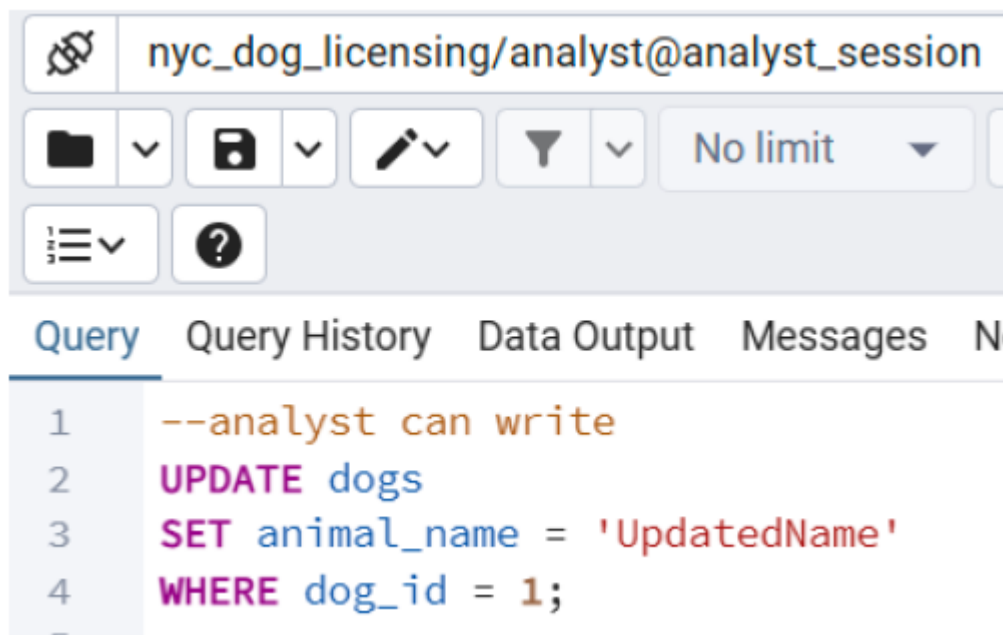
Query Query History Data Output Messages

ERROR: permission denied for table dogs

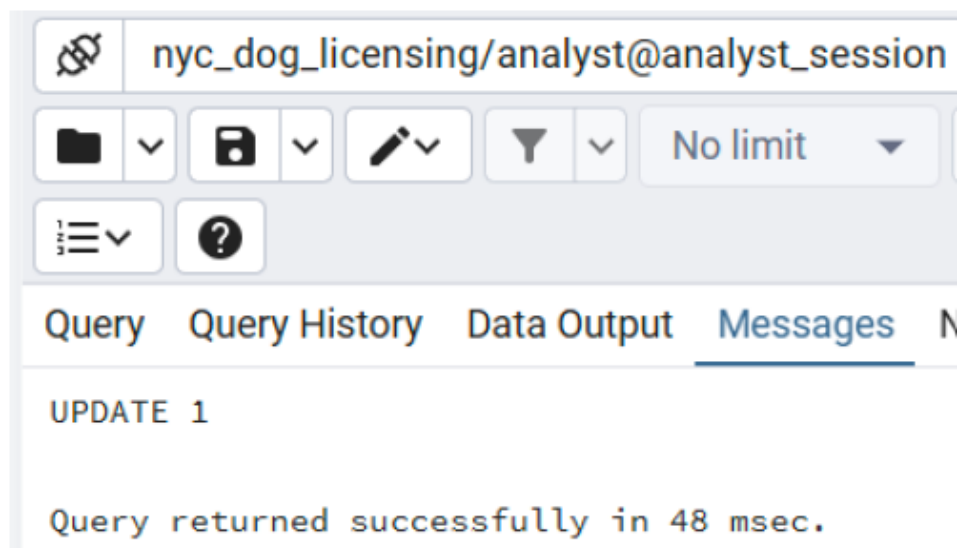
Now logged in as an analyst:



Analyst can write:



Proof of the result:



Part 2: Include short discussion of HIPAA/GLBA relevance to your dataset.

The NYC Dog Licensing dataset does not fall under HIPAA because there is no Protected Health Information (PHI) from a covered entity such as a hospital or insurer. It also does not fall under GLBA, which applies to financial institutions handling sensitive customer financial data. However, access-control best practices used in HIPAA and GLBA, such as least-privilege permissions, role separation, and preventing unauthorized modifications, are still relevant. Applying these principles ensures that potentially sensitive geographic or ownership-related data is handled responsibly, even if it is not legally regulated by HIPAA or GLBA. Zip codes are present in the dataset, and if any element of the dataset were combined with personal identifiers or medical or financial information about the owner or the animal, it could become sensitive and require stronger security controls.

NYC Dog Licensing Parts 2 & 4 Combined

December 7, 2025

```
[1]: !pip install psycpg2
    !pip install vega_datasets
    !pip install factor_analyzer
    !pip install SQLAlchemy psycpg2-binary pandas matplotlib seaborn !pip
    install factor_analyzer
    import psycpg2
    import pandas as pd
    import numpy as np
    import time
    from datetime import datetime
    import altair as alt
    import seaborn as sns
    alt.data_transformers.enable('json')
    import matplotlib.pyplot as plt
    from sqlalchemy import create_engine
    import re
    import unicode
    from factor_analyzer import FactorAnalyzer
    from factor_analyzer.factor_analyzer import calculate_kmo
    from sklearn.preprocessing import StandardScaler
    from sklearn.cluster import KMeans, AgglomerativeClustering
    from scipy.cluster.hierarchy import linkage, dendrogram
    from scipy.cluster.hierarchy import fcluster
    from sklearn.cluster import KMeans
    from sklearn.metrics import silhouette_score
```

Load the dog licensing dataset into a Pandas DataFrame

```
df = pd.read_csv('NYC_Dog_Licensing_Dataset_20251117.csv')
```

Display the first 5 rows

```
df.head()
```

Requirement already satisfied: psycopg2 in c:\users\adamk\anaconda3\lib\site packages (2.9.11)

Requirement already satisfied: vega_datasets in
c:\users\adamk\anaconda3\lib\site-packages (0.9.0)

Requirement already satisfied: pandas in c:\users\adamk\anaconda3\lib\site 1

packages (from vega_datasets) (2.3.2)

Requirement already satisfied: numpy>=1.26.0 in

c:\users\adamk\anaconda3\lib\site-packages (from pandas->vega_datasets) (2.2.6)

Requirement already satisfied: python-dateutil>=2.8.2 in

c:\users\adamk\anaconda3\lib\site-packages (from pandas->vega_datasets)

(2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in

c:\users\adamk\anaconda3\lib\site-packages (from pandas->vega_datasets) (2024.1)

Requirement already satisfied: tzdata>=2022.7 in

c:\users\adamk\anaconda3\lib\site-packages (from pandas->vega_datasets) (2025.2)

Requirement already satisfied: six>=1.5 in c:\users\adamk\anaconda3\lib\site packages (from
python-dateutil>=2.8.2->pandas->vega_datasets) (1.17.0) Requirement already satisfied:

factor_analyzer in

c:\users\adamk\anaconda3\lib\site-packages (0.5.1)

Requirement already satisfied: pandas in c:\users\adamk\anaconda3\lib\site packages
(from factor_analyzer) (2.3.2)

Requirement already satisfied: scipy in c:\users\adamk\anaconda3\lib\site packages
(from factor_analyzer) (1.16.3)

Requirement already satisfied: numpy in c:\users\adamk\anaconda3\lib\site packages
(from factor_analyzer) (2.2.6)

Requirement already satisfied: scikit-learn in

c:\users\adamk\anaconda3\lib\site-packages (from factor_analyzer) (1.6.1)

Requirement already satisfied: python-dateutil>=2.8.2 in

c:\users\adamk\anaconda3\lib\site-packages (from pandas->factor_analyzer)

(2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in

c:\users\adamk\anaconda3\lib\site-packages (from pandas->factor_analyzer) (2024.1)

Requirement already satisfied: tzdata>=2022.7 in

c:\users\adamk\anaconda3\lib\site-packages (from pandas->factor_analyzer) (2025.2)

Requirement already satisfied: six>=1.5 in c:\users\adamk\anaconda3\lib\site packages
(from python-dateutil>=2.8.2->pandas->factor_analyzer) (1.17.0) Requirement already

satisfied: joblib>=1.2.0 in

c:\users\adamk\anaconda3\lib\site-packages (from scikit-learn->factor_analyzer) (1.4.2)

Requirement already satisfied: threadpoolctl>=3.1.0 in

c:\users\adamk\anaconda3\lib\site-packages (from scikit-learn->factor_analyzer) (3.5.0)
Requirement already satisfied: SQLAlchemy in c:\users\adamk\anaconda3\lib\site packages (2.0.39)
Requirement already satisfied: psycpg2-binary in c:\users\adamk\anaconda3\lib\site-packages (2.9.11)
Requirement already satisfied: pandas in c:\users\adamk\anaconda3\lib\site packages (2.3.2)
Requirement already satisfied: matplotlib in c:\users\adamk\anaconda3\lib\site packages (3.10.0)
Requirement already satisfied: seaborn in c:\users\adamk\anaconda3\lib\site 2

packages (0.13.2)
Requirement already satisfied: greenlet!=0.4.17 in c:\users\adamk\anaconda3\lib\site-packages (from SQLAlchemy) (3.1.1)
Requirement already satisfied: typing-extensions>=4.6.0 in c:\users\adamk\anaconda3\lib\site-packages (from SQLAlchemy) (4.12.2)
Requirement already satisfied: numpy>=1.26.0 in c:\users\adamk\anaconda3\lib\site-packages (from pandas) (2.2.6)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\adamk\anaconda3\lib\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\adamk\anaconda3\lib\site-packages (from pandas) (2024.1)
Requirement already satisfied: tzdata>=2022.7 in c:\users\adamk\anaconda3\lib\site-packages (from pandas) (2025.2)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\adamk\anaconda3\lib\site-packages (from matplotlib) (1.3.1)
Requirement already satisfied: cycler>=0.10 in c:\users\adamk\anaconda3\lib\site-packages (from matplotlib) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\adamk\anaconda3\lib\site-packages (from matplotlib) (4.55.3)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\adamk\anaconda3\lib\site-packages (from matplotlib) (1.4.8)
Requirement already satisfied: packaging>=20.0 in c:\users\adamk\anaconda3\lib\site-packages (from matplotlib) (24.2)
Requirement already satisfied: pillow>=8 in c:\users\adamk\anaconda3\lib\site packages (from matplotlib) (11.1.0)
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\adamk\anaconda3\lib\site-packages (from matplotlib) (3.2.0)
Requirement already satisfied: six>=1.5 in c:\users\adamk\anaconda3\lib\site packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Requirement already satisfied: factor_analyzer in c:\users\adamk\anaconda3\lib\site-packages (0.5.1)
Requirement already satisfied: pandas in c:\users\adamk\anaconda3\lib\site packages (from factor_analyzer) (2.3.2)
Requirement already satisfied: scipy in c:\users\adamk\anaconda3\lib\site packages (from factor_analyzer) (1.16.3)
Requirement already satisfied: numpy in c:\users\adamk\anaconda3\lib\site packages (from factor_analyzer) (2.2.6)
Requirement already satisfied: scikit-learn in

c:\users\adamk\anaconda3\lib\site-packages (from factor_analyzer) (1.6.1)
 Requirement already satisfied: python-dateutil>=2.8.2 in
 c:\users\adamk\anaconda3\lib\site-packages (from pandas->factor_analyzer)
 (2.9.0.post0)
 Requirement already satisfied: pytz>=2020.1 in
 c:\users\adamk\anaconda3\lib\site-packages (from pandas->factor_analyzer) (2024.1)
 Requirement already satisfied: tzdata>=2022.7 in
 c:\users\adamk\anaconda3\lib\site-packages (from pandas->factor_analyzer) (2025.2)

3

Requirement already satisfied: six>=1.5 in c:\users\adamk\anaconda3\lib\site packages
 (from python-dateutil>=2.8.2->pandas->factor_analyzer) (1.17.0) Requirement already
 satisfied: joblib>=1.2.0 in
 c:\users\adamk\anaconda3\lib\site-packages (from scikit-learn->factor_analyzer) (1.4.2)
 Requirement already satisfied: threadpoolctl>=3.1.0 in
 c:\users\adamk\anaconda3\lib\site-packages (from scikit-learn->factor_analyzer) (3.5.0)
 C:\Users\adamk\AppData\Local\Temp\ipykernel_20656\2157518581.py:29: DtypeWarning:
 Columns (2) have mixed types. Specify dtype option on import or set low_memory=False.
 df = pd.read_csv('NYC_Dog_Licensing_Dataset_20251117.csv')

```
[1]: AnimalName AnimalGender AnimalBirthYear \
0 PAIGE F 2014
1 YOGI M 2010
2 ALI M 2014
3 QUEEN F 2013
4 LOLA F 2009
```

```
BreedName ZipCode LicenseIssuedDate \
0 American Pit Bull Mix / Pit Bull Mix 10035.0 09/12/2014 1 Boxer 10465.0
09/12/2014 2 Basenji 10013.0 09/12/2014 3 Akita Crossbreed 10013.0
09/12/2014 4 Maltese 10028.0 09/12/2014
```

```
LicenseExpiredDate Extract Year
0 09/12/2017 2016
1 10/02/2017 2016
2 09/12/2019 2016
3 09/12/2017 2016
4 10/09/2017 2016
```

```
[ ]:
```

Part 1 – Schema Design & Database Build C. Data-volume sanity check

- Short Python script that counts rows & reports NULL/invalid counts per table.

```
[2]: # Row count
print("Total rows:", len(df))
print("-" * 40)
```

```

# NULL counts
print("NULL counts per column:")
print(df.isnull().sum())
print("-" * 40)

```

4

```

# Basic invalid value checks
print("Invalid value checks:")

```

```

# Safely convert birth year to numeric
df['AnimalBirthYear'] = pd.to_numeric(df['AnimalBirthYear'], errors='coerce')

```

```

# Year should be reasonable (e.g., 1900–2024 as last updated 2023)
invalid_years = df[(df['AnimalBirthYear'] < 1900) | (df['AnimalBirthYear'] > 2024)]
invalid_years = invalid_years.dropna(subset=['AnimalBirthYear'])
print("Invalid birth years:", len(invalid_years))

```

```

# Missing ZIP codes
invalid_zip = df[df['ZipCode'].isnull()]
print("Missing ZIP codes:", len(invalid_zip))

```

```

# Missing license dates
invalid_issue_dates = df[df['LicenseIssuedDate'].isnull()]
invalid_expire_dates = df[df['LicenseExpiredDate'].isnull()]
print("Missing issue dates:", len(invalid_issue_dates))
print("Missing expiration dates:", len(invalid_expire_dates))

```

Total rows: 722864

NULL counts per column:

```

AnimalName 1810
AnimalGender 21
AnimalBirthYear 0
BreedName 0
ZipCode 9
LicenseIssuedDate 0
LicenseExpiredDate 82
Extract Year 0
dtype: int64

```

Invalid value checks:

```

Invalid birth years: 37
Missing ZIP codes: 9
Missing issue dates: 0
Missing expiration dates: 82

```


Part 2 – Data Ingestion & Preparation

A. ETL in Python

- Read raw files, handle encoding issues, convert data types, standardize units/dates.
- Insert into DB in chunks (COPY or executemany) with timing logs.

5

B. Data-quality notebook

- Missing-value profile, outlier detection, at least two transformations (e.g., binning, scaling, text cleaning), written justification for each.

In effort to insert clean and proper data into the DB in chunks, I will first focus on issues that are present with the data through exploration. Upon exposure to issues related to Part 2 A & B, I will make the necessary changes.

```
[3]: # Basic descriptive statistics for numeric columns
df.describe()
```

```
[3]: AnimalBirthYear ZipCode Extract Year
count 722834.000000 722855.000000 722864.000000
mean 2014.515215 10705.198814 2020.602199
std 13.849694 1074.167661 2.920664
min 1.000000 0.000000 2016.000000
25% 2011.000000 10031.000000 2017.000000
50% 2015.000000 10469.000000 2022.000000
75% 2019.000000 11228.000000 2023.000000
max 2024.000000 99508.000000 2024.000000
```

Per the descriptive statistics for the numeric columns, some issues are present. Per the readme included with this dataset, “this data provides information, on dog licenses in NYC, breed and license issue and expiration dates in a given calendar year, e.g. 2017 Dogs with Active Licenses data set contains active dog licenses in calendar year 2017 and was last updated on May 25th 2022.

The distribution of AnimalBirthYear shows a mix of reasonable values and clear issues. The mean is about 2014.5 and the median is 2015, which makes sense because most licensed dogs are fairly young and typically fall between 2011 and 2019. At the same time, the standard deviation is very high at around 13.85 years. That number is unrealistic for dog lifespans and immediately suggests that the dataset contains invalid or incorrectly entered birth years. The minimum value of 1 is impossible, and it is likely the result of a placeholder or a data entry error. The maximum value of 2024 is fine for puppies born recently, but the existence of extremely low or historically unlikely birth years confirms that this field needs cleaning before I can continue with any meaningful analysis.

The ZipCode column shows similar patterns. The mean and median values fall within the expected range for New York City ZIP codes, and the spread looks reasonable given how many neighborhoods exist across the city. However, the minimum value is 0, which is not a valid ZIP code. This points to at least some missing or miscoded data. The maximum value, 99508,

belongs to an address in Alaska. That may represent an out-of-area mailing address or another data entry mistake, but either way it is far outside the NYC range. Because of this, the ZIP code field also needs validation and cleanup, including flagging values like 0 and identifying entries that fall far outside the city.

In contrast, the Extract Year field looks clean and reliable. The mean is roughly 2020.6 and the median is 2022, which lines up well with the expected timeline of the NYC dog licensing dataset. Its standard deviation is small, and the values range from 2016 to 2024, which fits perfectly with the years the raw files were actually produced. Since this column does not appear to contain errors, it can be used as a stable reference when cleaning other fields, especially birth years.

Overall, these statistics make it clear that the main data quality issues are centered around

unreal 6

istic birth years and invalid ZIP codes. These will need to be addressed through cleaning, filtering, or imputation to make the dataset suitable for deeper analysis. The Extract Year field, on the other hand, appears consistent and does not require additional work.

```
[4]: # Keep only zip codes for NYC
```

```
df["ZipCode"] = pd.to_numeric(df["ZipCode"], errors="coerce")
df = df[df["ZipCode"].between(10000, 14999)]
df["ZipCode"] = df["ZipCode"].astype(int).astype(str).str.zfill(5) [5]: df.columns
```

```
[5]: Index(['AnimalName', 'AnimalGender', 'AnimalBirthYear', 'BreedName', 'ZipCode',
          'LicenseIssuedDate', 'LicenseExpiredDate', 'Extract Year'],
          dtype='object')
```

```
[6]: for col in df.columns:
      print(repr(col))
```

```
'AnimalName'
'AnimalGender'
'AnimalBirthYear'
'BreedName'
'ZipCode'
'LicenseIssuedDate'
'LicenseExpiredDate'
'Extract Year'
```

```
[7]: invalid_birth_years = df[
      (df['AnimalBirthYear'] < 1995) |
      (df['AnimalBirthYear'] > df['Extract Year'])
    ]

invalid_birth_years[['AnimalBirthYear', 'Extract Year']].head()
```

```
[7]: AnimalBirthYear Extract Year
3485 1991.0 2016
```

```
3542 1991.0 2016
3560 1991.0 2016
3590 1991.0 2016
3730 1991.0 2016
```

A quick check of the data shows cases like a birth year of 1991 paired with an extract year of 2016, which would imply a 25-year-old dog ($2016 - 1991 = 25$). This immediately raises a red flag because the typical dog lifespan is usually around 10–13 years, with only a small number of exceptional cases living significantly longer (Dickson). Values like this are not biologically realistic, so they should be treated as outliers or invalid entries during data cleaning.

```
[8]: # Convert the column to nullable integer before cleaning
     df['AnimalBirthYear'] = df['AnimalBirthYear'].astype('Int64')
```

7

```
[9]: test_outliers = df[
    (df['AnimalBirthYear'] < 1995) |
    (df['AnimalBirthYear'] > df['Extract Year']) |
    ((df['Extract Year'] - df['AnimalBirthYear']) > 30)
]

test_outliers.head()
```

```
[9]: AnimalName AnimalGender AnimalBirthYear BreedName ZipCode \
3485 REX M 1991
Siberian Husky 11385 3542 MABEL F 1991 Unknown 10011 3560 LUIGI M 1991 Jack Russell
Terrier 10454 3590 SCOUT M 1991 Cairn Terrier 10011 3730 YUM F 1991 Pomeranian 11373
```

```
LicenseIssuedDate LicenseExpiredDate Extract Year
3485 12/28/2015 01/30/2017 2016
3542 01/10/2016 01/13/2017 2016
3560 03/17/2016 04/30/2017 2016
3590 07/06/2016 07/30/2017 2016
3730 09/07/2016 09/30/2017 2016
```

```
[10]: # Convert birth year to nullable integer
      df['AnimalBirthYear'] = pd.to_numeric(df['AnimalBirthYear'], errors='coerce').astype('Int64')
```

```
[11]: # Identify and clean implausible birth years:
      # - BirthYear < 1995 (would imply dogs older than ~30 years)
      # - BirthYear > Extract Year (future birth year)
      # - Extract Year - BirthYear > 30 (biologically unrealistic age) # Any records matching
      these conditions are set to NA rather than deleted.
```

```
df.loc[
    (df['AnimalBirthYear'] < 1995) |
    (df['AnimalBirthYear'] > df['Extract Year']) |
    ((df['Extract Year'] - df['AnimalBirthYear']) > 30),
    'AnimalBirthYear'
```

```
] = pd.NA
```

```
# Display a sample of rows where the invalid birth years were replaced with NA  
df[df["AnimalBirthYear"].isna()].head()
```

```
[11]: AnimalName AnimalGender AnimalBirthYear BreedName ZipCode \ 3485 REX M <NA>  
Siberian Husky 11385 3542 MABEL F <NA> Unknown 10011 3560 LUIGI M <NA> Jack Russell  
Terrier 10454 3590 SCOUT M <NA> Cairn Terrier 10011
```

8

```
3730 YUM F <NA> Pomeranian 11373
```

```
LicenseIssuedDate LicenseExpiredDate Extract Year  
3485 12/28/2015 01/30/2017 2016  
3542 01/10/2016 01/13/2017 2016  
3560 03/17/2016 04/30/2017 2016  
3590 07/06/2016 07/30/2017 2016  
3730 09/07/2016 09/30/2017 2016
```

```
[12]: # Check for missingness  
df.isna().sum()
```

```
[12]: AnimalName 1790  
AnimalGender 21  
AnimalBirthYear 160  
BreedName 0  
ZipCode 0  
LicenseIssuedDate 0  
LicenseExpiredDate 54  
Extract Year 0  
dtype: int64
```

```
[13]: # Fill missing names with 'Unknown'  
df["AnimalName"] = df["AnimalName"].fillna("Unknown")
```

```
# This is a fairly large dataset, so i'm just going to drop some of the _ -missingness
```

```
# Drop rows where the expiration date is missing  
df = df.dropna(subset=["LicenseExpiredDate"])
```

```
# Dropping rows where birth year is unknown  
df = df.dropna(subset=["AnimalBirthYear"])
```

```
df = df.dropna(subset=["AnimalGender"])
```

```
#df = df.dropna(subset=["ZipCode"])
```

```
[14]: # Verify the missingness
      df.isna().sum()
```

```
[14]: AnimalName 0
      AnimalGender 0
      AnimalBirthYear 0
      BreedName 0
      ZipCode 0
      LicenseIssuedDate 0
```

9

```
LicenseExpiredDate 0
Extract Year 0
dtype: int64
```

ff Display the data types of each column. Which columns might need type conversion?

```
[15]: # Display the data types of each column
      print(df.dtypes)
```

```
AnimalName object
AnimalGender object
AnimalBirthYear Int64
BreedName object
ZipCode object
LicenseIssuedDate object
LicenseExpiredDate object
Extract Year int64
dtype: object
```

After examining the data types of my columns, several fields clearly require conversion to align with both the logical structure of the dataset and the SQL schema I designed earlier. The ZIP code column is currently stored as a float (e.g., 11385.0), which is inappropriate because ZIP codes function as identifiers rather than numeric values. This column needs to be converted to a string type so that it matches the VARCHAR(5) format used in the database. The date fields (LicenseIssuedDate and LicenseExpiredDate) are also stored as objects, meaning they are read in as plain text. Since the SQL schema requires these fields to be stored as DATE types and since I will need them for chronological comparisons later, both columns must be converted to proper datetime formats using `pd.to_datetime()`. The birth year and extract year fields are already integer-based, but for consistency with Pandas' nullable integer type, they should both be converted to Int64, which also ensures they can support null values after cleaning. The remaining object columns—such as AnimalName, AnimalGender, and BreedName—are already in an appropriate format since they map directly to text fields in the SQL schema. Converting these key columns ensures that the dataset is both analytically reliable and fully compatible with the database constraints established earlier.

```
[16]: # Convert ZIP Code from float to string
      # ZIP codes are identifiers, not numeric values.
```

```
df['ZipCode'] = df['ZipCode'].astype('Int64').astype('string')
```

```
# Convert date fields from object to datetime
```

```
# Required for SQL DATE type and chronological analysis.
```

```
df['LicenseIssuedDate'] = pd.to_datetime(df['LicenseIssuedDate'],   
    errors='coerce')
```

```
df['LicenseExpiredDate'] = pd.to_datetime(df['LicenseExpiredDate'],   
    errors='coerce')
```

10

```
# Convert BirthYear and Extract_Year to nullable Int64
```

```
# Ensures consistency and supports <NA> for cleaned values.
```

```
df['AnimalBirthYear'] = pd.to_numeric(df['AnimalBirthYear'], errors='coerce').astype('Int64')
```

```
df['Extract Year'] = pd.to_numeric(df['Extract Year'], errors='coerce').astype('Int64')
```

```
# Display updated dtypes to verify conversion
```

```
df.dtypes
```

[16]: AnimalName object

AnimalGender object

AnimalBirthYear Int64

BreedName object

ZipCode string[python]

LicenseIssuedDate datetime64[ns]

LicenseExpiredDate datetime64[ns]

Extract Year Int64

dtype: object

After performing the conversions, the resulting data types now align with the SQL schema and support reliable analysis. ZIP codes have been converted to string types, preventing formatting issues such as dropped leading zeros. Both license date fields are now proper datetime objects, enabling chronological comparisons and seamless loading into SQL DATE columns. The birth year and extract year fields use Pandas' nullable Int64 type, allowing values without breaking integer operations. The remaining text fields remain as object types, which map directly to SQL TEXT. Overall, the dataset is now structurally clean and ready for further data-quality checks and database insertion after cleaning and standardizing the dog names, as well as breed names.

```
[17]: # Identify rows where the license expired before it was issued invalid_date_rows =   
    df[df['LicenseIssuedDate'] > df['LicenseExpiredDate']]
```

```
invalid_date_rows.head()
```

[17]: Empty DataFrame

Columns: [AnimalName, AnimalGender, AnimalBirthYear, BreedName, ZipCode, LicenseIssuedDate, LicenseExpiredDate, Extract Year]
Index: []

The check for logical date consistency returned no results, meaning there are no cases in which a license was issued after it expired. Since LicenseIssuedDate is always earlier than or equal to LicenseExpiredDate, the dataset does not require any corrections for date order. This confirms that the date fields are not only formatted correctly but also historically consistent, which supports accurate time-based analysis and ensures compatibility with SQL constraints.

Cleaning/Standardizing Dog Information:

11

[18]: *# Convert all animal names to title case*

```
df['AnimalName'] = df['AnimalName'].str.title()
```

Display the first few rows to verify that the formatting was applied correctly

```
df[['AnimalName']].head()
```

[18]: AnimalName

0 Paige

1 Yogi

2 Ali

3 Queen

4 Lola

ff Check if dog names have unexpected formatting similar to our mini-projectect (e.g., “mrs. jEsslCa BIAck md”). Clean these names by removing anyhting abnormal.

[19]: *# Find names containing characters other than letters, periods, hyphens, or _ -spaces*

```
weird_names = df[df['AnimalName'].str.contains(r"^[^A-Za-z\.\-\_ ]$", na=False)]
```

Show first 20 abnormal names

```
weird_names.head(20)
```

[19]: AnimalName AnimalGender AnimalBirthYear \

2037 Bluewave'S F 2014

2090 It'S F 2013

2704 Creekside'S M 2014

2767 Reese'S F 2014

2833 Kuk'Uk M 2014

3034 Ogden16668022 M 2004

4222 Rösti F 2013

4506 Chiquitín M 2010

4963 Kuz'Ma M 2012

5075 Cafe' M 2014

5117 D'Artagnan M 2012

5908 Ca\$H M 2013

5961 Stewie(Berman) M 2009

6015 Bella/Tinkerbelle F 2012
6212 Bonita, F 2014
6499 Béatrice F 2014
6687 Badamtam'S F 2010
7490 Xöe F 2012
8309 Dunkin' M 2000
8527 Ncr6627 F 2014

BreedName ZipCode LicenseIssuedDate \
2037 Collie, Smooth Coat 10028 2015-01-09 2090 Briard 10025 2015-01-10

12

2704 Labrador Retriever 10025 2015-01-18 2767 Maltese 11226 2015-01-19
2833 American Eskimo dog 11201 2015-01-20 3034 Siberian Husky 10014
2015-01-22 4222 St. Bernard 11205 2015-02-07 4506 Chihuahua 10468
2015-02-11 4963 Welsh Corgi, Pembroke 10025 2015-02-17 5075 American Pit
Bull Mix / Pit Bull Mix 11207 2015-02-18 5117 Pomeranian 11106 2015-02-18
5908 American Pit Bull Terrier/Pit Bull 10458 2015-02-27 5961 Unknown 11214
2015-02-27 6015 Unknown 10027 2015-02-28 6212 Beagle Crossbreed 11226
2015-03-03 6499 American Pit Bull Mix / Pit Bull Mix 11216 2015-03-05 6687
Dachshund, Long Haired 11217 2015-03-08 7490 Rat Terrier 11211 2015-03-18
8309 West High White Terrier 11367 2015-03-26 8527 Unknown 10306
2015-03-27

LicenseExpiredDate Extract Year
2037 2016-01-09 2016
2090 2016-01-10 2016
2704 2016-01-18 2016
2767 2016-01-19 2016
2833 2016-01-20 2016
3034 2017-01-22 2016
4222 2016-02-07 2016
4506 2016-02-11 2016
4963 2016-03-25 2016
5075 2016-02-18 2016
5117 2016-02-18 2016
5908 2017-02-11 2016
5961 2016-03-25 2016
6015 2016-03-23 2016
6212 2016-03-03 2016
6499 2016-03-05 2016
6687 2016-03-08 2016
7490 2016-03-18 2016
8309 2016-03-30 2016
8527 2016-03-27 2016

[20]: df.loc[df['AnimalName'] == 'Ca\$H', 'AnimalName_Clean'] = 'Cash'


```
[21]: def clean_animal_name(name):
```

```
    # Handle missing values
```

```
    if pd.isna(name):
```

```
        return name
```

13

```
    # Normalize accents (Café → Cafe, José → Jose)
```

```
    name = unidecode.unidecode(str(name))
```

```
    # Detect and normalize initials (A.J., T.J.)
```

```
    # Pattern: A. or A.J. or A.J.K.
```

```
    if re.fullmatch(r"([A-Za-z]\.)+[A-Za-z]?\.?", name.strip()):
```

```
        # Remove periods and uppercase (A.J. → AJ)
```

```
        return name.replace(".", "").upper()
```

```
    # Add spacing after titles (Mr., Mrs., Dr.)
```

```
    name = re.sub(
```

```
        r"\b(Mr|Mrs|Ms|Miss|Dr)(?=[\.\s])\.",
```

```
        r"\1 ",
```

```
        name,
```

```
        flags=re.IGNORECASE
```

```
)
```

```
    # Convert possessive endings before removing apostrophes (Reese's Reeses) name =
```

```
    re.sub(r"(?i)'s$", "s", name)
```

```
    # Remove remaining apostrophes (D'Artagnan → Dartagnan)
```

```
    name = name.replace("'", "")
```

```
    # Remove unwanted characters except letters, hyphens, and spaces name
```

```
= re.sub(r"[^A-Za-z\-\s]", "", name)
```

```
    # Remove periods (D.G. to DG)
```

```
    name = name.replace(".", "")
```

```
    # Collapse multiple spaces
```

```
    name = re.sub(r"\s+", " ", name).strip()
```

```
    # Title case ("mr rex" to "Mr Rex")
```

```
    name = name.title()
```

```
    return name
```

```
[22]: # Apply the custom 'clean_animal_name' function to every row in the _
```

```
      ↪ 'AnimalName' column
```

```
df['AnimalName_Clean'] = df['AnimalName'].apply(clean_animal_name)
```

[23]: # Display the first 50 rows where the original animal name differs from the cleaned version

```
df[df['AnimalName'] != _  
df['AnimalName_Clean']] [['AnimalName', 'AnimalName_Clean']].head(50) 14
```

[23]: AnimalName AnimalName_Clean 83 A.J. AJ 1190
Mr. Mr 1266 Miss. Miss 1275 T.J. TJ 1293
Mr.Pickles Mr Pickles 1348 Dr. Dr 1959 Mr. Mr
1967 J.D JD 1990 Mr. Mr 2037 Bluewave'S
Bluewaves 2090 It'S Its 2091 Miss.Wiggles Miss
Wiggles 2400 C.J. CJ 2704 Creekside'S
Creeksides 2767 Reese'S Reeses 2833 Kuk'Uk
Kukuk 2934 J.D. JD 2980 K.C. KC 3034
Ogden16668022 Ogden 3063 Mr. Mr 3270 O.P.
OP 3439 A.J. AJ 3905 Jr. Jr 4222 Rösti Rosti
4506 Chiquitin Chiquitin 4963 Kuz'Ma Kuzma
5072 M.J. MJ 5075 Cafe' Cafe 5117 D'Artagnan
Dartagnan 5908 Ca\$H Ca H 5961
Stewie(Berman) Stewie Berman 6015
Bella/Tinkerbelle Bella Tinkerbelle 6186 Mrs. Mrs
6212 Bonita, Bonita 6430 Mr. Mr 6499 Béatrice
Beatrice 6687 Badamtam'S Badamtams 7490
Xöe Xoe 8309 Dunkin' Dunkin 8474 Dr. Dr 8527
Ncr6627 Ncr 8562 Mr. Mr 8589 Mohogany/
Mohogany 8899 Mr. Mr 9147 Gizmo/Gilb Gizmo
Gilb 9366 Mr. Mr

15

9417 Mr. Mr
9434 K9 K
9496 J.J. JJ
9603 Capt. Capt

[24]: # Fix where the CLEANED name is wrong, not the raw name

```
df.loc[df['AnimalName_Clean'] == 'Ca H', 'AnimalName_Clean'] = 'Cash'
```

ff Standardize the “BreedName” field by ensuring consistent capitalization, spacing, and naming conventions. Identify and resolve common inconsistencies such as mixed case formatting, and irregular punctuation.

[25]: df['BreedName'].unique()[:50]

[25]: array(['American Pit Bull Mix / Pit Bull Mix', 'Boxer', 'Basenji', 'Akita Crossbreed',
'Maltese', 'Unknown',
'Labrador Retriever Crossbreed', 'Dachshund Smooth Coat',
'Bull Dog, French', 'Miniature Pinscher', 'Yorkshire Terrier',

```
'Dachshund Smooth Coat Miniature', 'German Shepherd Dog',
'Cavalier King Charles Spaniel',
'American Pit Bull Terrier/Pit Bull', 'Beagle', 'Havanese',
'Poodle, Standard', 'German Shorthaired Pointer', 'Bull Terrier',
'Dachshund, Long Haired', 'Schnauzer, Miniature',
'Labrador Retriever', 'Boston Terrier', 'Norfolk Terrier',
'Bichon Frise', 'Golden Retriever', 'Chihuahua', 'Collie, Border', 'Poodle,
Miniature', 'Shih Tzu', 'Kooikerhondje',
'Bull Dog, English', 'American Eskimo dog', 'Collie, Smooth Coat', 'Cocker
Spaniel', 'Puggle', 'Lhasa Apso', 'French Bulldog',
'Siberian Husky', 'Poodle', 'Poodle, Toy', 'Jack Russell Terrier', 'Bulldog',
'Labradoodle', 'Shiba Inu', 'Pug',
'Dachshund, Long Haired Miniature',
'American Staffordshire Terrier', 'Tibetan Terrier'], dtype=object) [26]:
```

```
df['BreedName'].nunique()
```

[26]: 1595

```
[27]: def clean_breed_name(breed):
    if pd.isna(breed):
        return breed

    # Convert accents (rare but possible)
    breed = unidecode.unidecode(str(breed))

    # Lowercase for consistent processing
    b = breed.lower().strip()

    # Normalize hyphens to spaces
```

16

```
b = re.sub(r"[\-\_\—]", " ", b)

    # Normalize slashes to a single slash
    b = re.sub(r"[W]", "/", b)

    # Remove stray punctuation (commas, quotes, parentheses)
    b = re.sub(r"[\(\),]", " ", b)

    # Collapse multiple spaces
    b = re.sub(r"\s+", " ", b).strip()

    # Title case for final output
    b = b.title()

    return b
```

```
# Apply the cleaning function
```

```
df['BreedName_Clean'] = df['BreedName'].apply(clean_breed_name)
```

```
# Preview before & after
```

```
df[['BreedName', 'BreedName_Clean']].head(20)
```

```
[27]: BreedName BreedName_Clean 0 American Pit Bull Mix / Pit Bull Mix American Pit Bull Mix /  
Pit Bull Mix 1 Boxer Boxer 2 Basenji Basenji 3 Akita Crossbreed Akita Crossbreed 4 Maltese  
Maltese 5 Unknown Unknown 6 Unknown Unknown 7 Labrador Retriever Crossbreed  
Labrador Retriever Crossbreed 8 Dachshund Smooth Coat Dachshund Smooth Coat 9 Bull  
Dog, French Bull Dog French 10 Miniature Pinscher Miniature Pinscher 11 Yorkshire Terrier  
Yorkshire Terrier 12 Dachshund Smooth Coat Miniature Dachshund Smooth Coat Miniature  
13 German Shepherd Dog German Shepherd Dog 14 Unknown Unknown 15 American Pit  
Bull Mix / Pit Bull Mix American Pit Bull Mix / Pit Bull Mix 16 Cavalier King Charles Spaniel  
Cavalier King Charles Spaniel 17 Cavalier King Charles Spaniel Cavalier King Charles  
Spaniel 18 American Pit Bull Terrier/Pit Bull American Pit Bull Terrier/Pit Bull 19 Boxer Boxer
```

```
[28]: breed_replacements = {
```

```
    # Reverse comma-breeds
```

```
    "Bull Dog French": "French Bulldog",
```

17

```
    "Bull Dog English": "English Bulldog",
```

```
    "Schnauzer Miniature": "Miniature Schnauzer",
```

```
    "Collie Border": "Border Collie",
```

```
    "Dachshund Long Haired": "Long-Haired Dachshund",
```

```
    "Dachshund Smooth Coat Miniature": "Miniature Smooth-Coat Dachshund",
```

```
    "Shar Pei Chinese": "Chinese Shar-Pei",
```

```
    "Chinese Shar Pei": "Chinese Shar-Pei",
```

```
    # Pit bull variants
```

```
    "Pitbull": "Pit Bull",
```

```
    "Pit Bull Mix": "Pit Bull Mix",
```

```
    "American Pit Bull Mix": "American Pit Bull Mix",
```

```
    # Common synonyms
```

```
    "Poodle Standard": "Standard Poodle",
```

```
    "Poodle Miniature": "Miniature Poodle",
```

```
    "Poodle Toy": "Toy Poodle",}
```

```
[29]: # unique cleaned breed names
```

```
    unique_breeds = sorted(df['BreedName_Clean'].dropna().unique())
```

```
    len(unique_breeds)
```

```
[29]: 1346
```

```
[30]: # Detect mix patterns
```

```

mix_breeds = [b for b in unique_breeds
               if ("mix" in b.lower()
                   or "cross" in b.lower()
                   or " x " in b.lower()
                   or "/" in b)]
mix_breeds[:20]

```

```

[30]: ['Affenpinscher Crossbreed',
       'Afghan Hound Crossbreed',
       'Airedale Terrier Crossbreed',
       'Akita Crossbreed',
       'Akita Mix',
       'Alaskan Malamute Crossbreed',
       'Am Eskimo / Maltese',
       'Am Pit Bull Mix',
       'Am Pit Bull Terrier Mix',
       'Am Stafford Terrier Mix',
       'Amer Pit Bull Terr Mix',
       'Amer.Eskimo Dog Mix',
       'American Bull Dog Crossbreed',
       'American Eskimo Dog Crossbreed',
       'American Pit Bull Mix / Pit Bull Mix',

```

18

```

       'American Pit Bull Terrier/Pit Bull',
       'American Spaniel Mix',
       'Aus Shep/Heeler/Ret',
       'Aus Shepherd/Border Collie Mix',
       'Aust Heller Mix']

```

[31]: # *Detect doodle/poo/poodle-hybrid patterns*

```

doodle_breeds = [b for b in unique_breeds
                  if ("doodle" in b.lower() or "poo" in b.lower())]
doodle_breeds[:20]

```

```

[31]: ['America Eskimo And Poodle',
       'Australian Labradoodle',
       'Bernedoodle',
       'Bichon Frise/Poodle Terrier',
       'Bichon Poo',
       'Bichon Poodle',
       'Bichon Poodle Mix',
       'Bichon/Poodle',
       'Bichon/Poodle Mix',
       'Biuchon/Poodle Mix',

```

```
'Cava Poo',
'Cavapoo',
'Chi Poo',
'Chih/Poo',
'Chihuahua /Poodle',
'Coca Poo',
'Cock A Poo',
'Cocka Poo',
'Cockapoo',
'Cocker Poo']
```

[32]: # *Detect hyphens or special separators*

```
hyphen_breeds = [b for b in unique_breeds if "-" in b]
hyphen_breeds[:20]
```

[32]: []

Breed standardization

[33]: # --- *helpers* ---

```
ABBREV = {
    r"\bam\b": "American",
    r"\bamer\b": "American",
    r"\bamer\.b": "American",
    r"\baus\b": "Australian",
```

19

```
    r"\baust\b": "Australian",
    r"\beng\b": "English",
    r"\bfr\b": "French",
    r"\bgerm\b": "German",
}
```

```
REPLACEMENTS_CANON = {
```

```
    # Comma-reversed & common synonyms
```

```
    "Bull Dog French": "French Bulldog",
    "Bull Dog English": "English Bulldog",
    "Schnauzer Miniature": "Miniature Schnauzer",
    "Collie Border": "Border Collie",
    "Poodle Standard": "Standard Poodle",
    "Poodle Miniature": "Miniature Poodle",
    "Poodle Toy": "Toy Poodle",
    "Shar Pei Chinese": "Chinese Shar Pei",
    "Chinese Shar Pei": "Chinese Shar-Pei",
    "Flat Coated Retriever": "Flat-Coated Retriever", "Curly
    Coated Retriever": "Curly-Coated Retriever", }
```

```

def expand_abbrev(s: str) -> str:
    txt = " " + s + " "
    for pat, repl in ABBREV.items():
        txt = re.sub(pat, f"{repl}", txt, flags=re.IGNORECASE)
    return re.sub(r"\s+", " ", txt).strip()

def fix_comma_breed(s: str) -> str:
    # Reverse simple "Breed, Descriptor" → "Descriptor Breed" if "," in
    s and s.count(",") == 1:
        a, b = [p.strip() for p in s.split(",", 1)]
        if a and b:
            return f"{b} {a}"
    return s

def standardize_poodle_sizes(s: str) -> str:
    low = s.lower()
    if "poodle" in low:
        if "standard" in low:
            return "Standard Poodle"
        if "mini" in low:
            return "Miniature Poodle"
        if "toy" in low:
            return "Toy Poodle"
    return s

def standardize_doodle_family(s: str) -> str:

```

20

```

    low = s.lower()
    # Doodles
    if "dood" in low:
        if "gold" in low:
            return "Goldendoodle"
        if "labra" in low:
            return "Labradoodle"
        if "berne" in low:
            return "Bernedoodle"
        if "aussie" in low or "australian shepherd" in low:
            return "Aussiedoodle"
        if "shep" in low:
            return "Shepadoodle"
    return s # unknown doodle → leave
    # Poo family
    if "poo" in low:
        if "york" in low:
            return "Yorkie Poo"

```

```

    if "multi" in low:
        return "Maltipoo"
    if "cock" in low or "cocker" in low:
        return "Cockapoo"
    if "shih" in low:
        return "Shih Poo"
    if "chi" in low:
        return "Chi Poo"
    if "bichon" in low:
        return "Bichon Poo"
    if "cava" in low:
        return "Cavapoo"
return s

```

```

def normalize_mix_markers(s: str) -> str:
    # Normalize various markers to a common structure
    # 1) unify separators
    s = s.replace("x", " x ").replace(" X ", " x ").replace(" x ", " / ") s =
    re.sub(r"\s*[\\V]\s*", " / ", s) # slash with spaces
    # 2) cross/crossbreed → Mix
    s = re.sub(r"\b(crossbreed|crossed|cross)\b", "Mix", s, flags=re.IGNORECASE) # 3) terrier
    X, shepherd x, etc. → "Breed Mix"
    s = re.sub(r"\b([A-Za-z][A-Za-z\s]+)\s+X\b", r"\1 Mix", s, flags=re.
    IGNORECASE)
    # Collapse spaces
    s = re.sub(r"\s+", " ", s).strip()
return s

```

```

def canonicalize_slash_mix(s: str) -> str:

```

21

"""If it's a slash combo (Breed1 / Breed2 ...), alphabetize parts and append 'Mix' once."""

```

    if " / " not in s:
        return s
    parts = [p.strip() for p in s.split(" / ") if p.strip()]
    if len(parts) < 2:
        return s
    parts = sorted(set(parts), key=lambda x: x.lower())
    return " / ".join(parts) + " Mix"

```

```

def final_titlecase_and_replacements(s: str) -> str:
    out = re.sub(r"\s+", " ", s).strip()
    out = out.title()
    # Apply canonical synonym fixes
    if out in REPLACEMENTS_CANON:
        out = REPLACEMENTS_CANON[out]

```



```

    # Normalize specific families post-title
    out = standardize_poodle_sizes(out)
    if out in REPLACEMENTS_CANON:
        out = REPLACEMENTS_CANON[out]
    return out

def clean_breed_medium(breed):
    if pd.isna(breed):
        return breed
    b = unicode.unidecode(str(breed)).strip()

    # expand common abbreviations
    b = expand_abbrev(b)

    # fix simple comma reversals
    b = fix_comma_breed(b)

    # normalize mix markers (X / Cross / Crossbreed / slashes) b =
    normalize_mix_markers(b)

    # doodle/poo families
    b = standardize_doodle_family(b)

    # if multi-part slash, canonicalize to alphabetized "Breed1 / Breed2 Mix" b =
    canonicalize_slash_mix(b)

    # final formatting & canonical replacements
    b = final_titlecase_and_replacements(b)

    # last small normalizations
    b = b.replace(" ", "").strip()

    22

    # unify "Terrier X" leftover to "Terrier Mix"
    b = re.sub(r"\bX\b", "Mix", b)

    return b

```

```
[34]: df["BreedName_Final"] = df["BreedName_Clean"].apply(clean_breed_medium)
```

```
[35]: # How many unique breeds after medium standardization?
```

```
df["BreedName_Final"].nunique()
```

```
# Which final labels still came from many messy originals? (
```

```
df.groupby("BreedName_Final")["BreedName_Clean"]
.nunique()
.sort_values(ascending=False)
```

```

        .head(30)
    )

    # Spot check changes
    df[df["BreedName_Final"] != df["BreedName_Clean"]][
        ["BreedName", "BreedName_Clean", "BreedName_Final"]
    ].sample(20, random_state=1)

    # See top 30 breeds now (clean)
    df["BreedName_Final"].value_counts().head(30)

```

```

[35]: BreedName_Final
Unknown 63526
Yorkshire Terrier 40174
Shih Tzu 37064
Chihuahua 28071
Labrador Retriever 21829
Maltese 20833
Labrador Retriever Mix 19322
American Pit Bull Mix / Pit Bull Mix Mix 17249
Shih Tzu Mix 14542
Terrier Mix 14072
Pomeranian 13961
French Bulldog 13435
Poodle Mix 13423
American Pit Bull Terrier / Pit Bull Mix 13114
Havanese 12247
Chihuahua Mix 11581
Golden Retriever 10883
Goldendoodle 10306
Beagle 10168
German Shepherd Dog 10105

```

23

```

Poodle 9159
Shiba Inu 8809
Cavalier King Charles Spaniel 8517
Miniature Schnauzer 8127
Jack Russell Terrier 8059
Toy Poodle 7665
Siberian Husky 7412
Yorkshire Terrier Mix 7289
German Shepherd Mix 7206
Miniature Poodle 7003
Name: count, dtype: int64

```

```
[36]: def remove_double_mix(s):
        # Remove trailing 'Mix Mix'
        s = re.sub(r"Mix\s+Mix$", "Mix", s)
        # Remove cases like 'Terrier Mix Mix'
        s = re.sub(r"(\bMix\b)s+(\bMix\b)", r"1", s)
        return s

df["BreedName_Final"] = df["BreedName_Final"].apply(remove_double_mix)

[37]: df[df["BreedName_Final"].str.contains(r"^\d+$|^\?\bMix Mix\b", na=False)]\
      ["BreedName",
       "BreedName_Clean", "BreedName_Final"].head(20)

[37]: BreedName BreedName_Clean BreedName_Final
463682 Mixed / ? Mixed / ? ? / Mixed Mix
464328 6152012 6152012 6152012

[38]: fix_mapping = {
        "6152012": "Unknown",
        "? / Mixed Mix": "Mixed Breed",
        "?": "Unknown",
        "? Mix": "Mixed Breed",
        "Mixed / ?": "Mixed Breed",
    }

df["BreedName_Final"] = df["BreedName_Final"].replace(fix_mapping) [39]:

df["BreedName_Final"].value_counts().head(30)

[39]: BreedName_Final
Unknown 63527
Yorkshire Terrier 40174
Shih Tzu 37064
Chihuahua 28071
Labrador Retriever 21829
Maltese 20833
```

24

```
Labrador Retriever Mix 19322
American Pit Bull Mix / Pit Bull Mix 17249
Shih Tzu Mix 14542
Terrier Mix 14072
Pomeranian 13961
French Bulldog 13435
Poodle Mix 13423
American Pit Bull Terrier / Pit Bull Mix 13114
Havanese 12247
```

Chihuahua Mix 11581
 Golden Retriever 10883
 Goldendoodle 10306
 Beagle 10168
 German Shepherd Dog 10105
 Poodle 9159
 Shiba Inu 8809
 Cavalier King Charles Spaniel 8517
 Miniature Schnauzer 8127
 Jack Russell Terrier 8059
 Toy Poodle 7665
 Siberian Husky 7412
 Yorkshire Terrier Mix 7289
 German Shepherd Mix 7206
 Miniature Poodle 7003
 Name: count, dtype: int64

[40]: # *Check for trailing space*

```
df[df["BreedName_Final"].str.contains(r"\s+$", na=False)][["BreedName",
~"BreedName_Final"]].head(20)
```

[40]: Empty DataFrame

Columns: [BreedName, BreedName_Final]
Index: []

[41]: # *Check for leading space*

```
df[df["BreedName_Final"].str.contains(r"^s+", na=False)][["BreedName",
~"BreedName_Final"]].head(20)
```

[41]: Empty DataFrame

Columns: [BreedName, BreedName_Final]
Index: []

[42]: # *Check for double or abnormal spacing*

```
df[df["BreedName_Final"].str.contains(r"\s{2,}", na=False)][["BreedName",
~"BreedName_Final"]].head(20)
```

25

[42]: Empty DataFrame

Columns: [BreedName, BreedName_Final]
Index: []

[43]: # *Post-cleaning mapping*

```
BREED_FINAL_MAP = {
    "German Shepherd Dog": "German Shepherd",
```

```

    "Standard Poodle": "Poodle",
    "Miniature Poodle": "Poodle",
    "Toy Poodle": "Poodle",
    "American Pit Bull Terrier": "American Pit Bull",
    "American Pit Bull Mix": "American Pit Bull Mix",
    "Pit Bull Mix": "American Pit Bull Mix",
    "Unknown": pd.NA,
    "Mixed": pd.NA,
}

```

[44]: `df['BreedName_Final'] = df['BreedName_Final'].replace(BREED_FINAL_MAP)` [45]:

`df['BreedName_Final'].value_counts().head(30)`

```

[45]: BreedName_Final
Yorkshire Terrier 40174
Shih Tzu 37064
Chihuahua 28071
Poodle 27729
Labrador Retriever 21829
Maltese 20833
Labrador Retriever Mix 19322
American Pit Bull Mix / Pit Bull Mix 17249
Shih Tzu Mix 14542
Terrier Mix 14072
Pomeranian 13961
French Bulldog 13435
Poodle Mix 13423
American Pit Bull Terrier / Pit Bull Mix 13114
Havanese 12247
Chihuahua Mix 11581
Golden Retriever 10883
Goldendoodle 10306
Beagle 10168
German Shepherd 10106
Shiba Inu 8809
Cavalier King Charles Spaniel 8517
Miniature Schnauzer 8127
Jack Russell Terrier 8059
Siberian Husky 7412

```

```

Yorkshire Terrier Mix 7289
German Shepherd Mix 7206
Maltese Mix 6949
Australian Shepherd 6848

```

Cocker Spaniel 6700
Name: count, dtype: int64

After reviewing the cleaned breed names, I noticed that a few Pit Bull related categories were still fragmented into multiple forms such as “American Pit Bull Mix / Pit Bull Mix” and “American Pit Bull Terrier / Pit Bull Mix.” These entries describe the same mixed-breed type, so I applied a final mapping step to consolidate them into a single label, “American Pit Bull Mix.” This reduces redundancy, simplifies the breed distribution, and ensures consistent categorization for analysis and database loading.

```
[46]: df['BreedName_Final'] = df['BreedName_Final'].replace({
        "American Pit Bull": "American Pit Bull",
        "American Pit Bull Mix": "American Pit Bull Mix",
        "Pit Bull Mix": "American Pit Bull Mix",
        "Pit Bull": "American Pit Bull",
    })

[47]: # Check if any cleaned breed still maps to >5 raw inputs
df.groupby("BreedName_Final")["BreedName_Clean"].nunique().
    sort_values(ascending=False).head(20)
```

```
[47]: BreedName_Final
Shih Poo 18
Yorkie Poo 17
Cockapoo 11
Poodle 11
Bichon Poo 7
Maltese / Shih Tzu Mix 6
Labradoodle 6
Goldendoodle 6
Maltese / Yorkie Mix 5
Shih Tzu / Yorkie Mix 4
Maltese Mix 3
Jack Russell Mix 3
Beagle / Dachshund Mix 3
Rat Terrier Mix 3
American Pit Bull Mix 3
Schnauzer Mix 3
Boxer Mix 3
Shih Tzu Mix 3
Border Collie Mix 3
Dachshund Mix 3
Name: BreedName_Clean, dtype: int64
```

```
[48]: # Verify nulls
df.isna().sum()
```

```
[48]: AnimalName 0
      AnimalGender 0
      AnimalBirthYear 0
      BreedName 0
      ZipCode 0
      LicenseIssuedDate 0
      LicenseExpiredDate 0
      Extract Year 0
      AnimalName_Clean 0
      BreedName_Clean 0
      BreedName_Final 63647
      dtype: int64
```

```
[49]: # Replace Missing BreedName_Final with "Unknown"
      df['BreedName_Final'] = df['BreedName_Final'].fillna("Unknown")
```

Justification here is while there are 63,647 nulls, the other available information can still provide insights, and in my opinion is not worth dropping.

```
[50]: # Create a lookup table of unique breeds
      unique_breeds = sorted(df['BreedName_Final'].unique())
```

```
breeds_df = pd.DataFrame({
    'breed_name': unique_breeds,
})
```

```
# Assign surrogate keys
```

```
breeds_df['breed_id'] = range(1, len(breeds_df) + 1)
```

```
breeds_df.head()
```

```
[50]: breed_name breed_id
      0 Affenpinscher 1
      1 Affenpinscher Mix 2
      2 Afghan Hound 3
      3 Afghan Hound Mix 4
      4 Airedale Terrier 5
```

```
[51]: # Fix known typo
```

```
breeds_df.loc[breeds_df['breed_name'] == "Aierdale Terrier", 'breed_name'] = "Airedale Terrier"
```

```
[52]: # Sort the breeds alphabetically to keep the lookup table organized # reset_index(drop=True)
      removes the old index and replaces it with a clean 0.. N-1 index
```

```
breeds_df = breeds_df.sort_values(by='breed_name').reset_index(drop=True)
```

```

# Reassign breed_id values so they match the new sorted order
# range(1, len(...) + 1) creates sequential IDs starting at 1
breeds_df['breed_id'] = range(1, len(breeds_df) + 1)

# Display the first few rows to verify the new sorted order and IDs
breeds_df.head()

```

```

[52]: breed_name breed_id
      0 Affenpinscher 1
      1 Affenpinscher Mix 2
      2 Afghan Hound 3
      3 Afghan Hound Mix 4
      4 Airedale 5

```

```

[53]: # Map BreedName_Final to breed_id
      breed_map = dict(zip(breeds_df['breed_name'], breeds_df['breed_id']))
      df['breed_id'] = df['BreedName_Final'].map(breed_map)

```

```

[54]: # Only 1 missing, going to drop
      df = df.dropna(subset=['breed_id'])

```

(Repeat) Part 1 – Schema Design & Database Build C. Data-volume sanity check

- Short Python script that counts rows & reports NULL/invalid counts per table.

```

[55]: # Row count
      print("Total rows:", len(df))
      print("-" * 40)

      # NULL counts
      print("NULL counts per column:")
      print(df.isnull().sum())
      print("-" * 40)

      # Basic invalid value checks
      print("Invalid value checks:")

      # Safely convert birth year to numeric
      df['AnimalBirthYear'] = pd.to_numeric(df['AnimalBirthYear'], errors='coerce')

      # Year should be reasonable (e.g., 1900–2024 as last updated 2023)
      invalid_years = df[(df['AnimalBirthYear'] < 1900) | (df['AnimalBirthYear'] > 2024)]
      invalid_years = invalid_years.dropna(subset=['AnimalBirthYear'])
      print("Invalid birth years:", len(invalid_years))

```



```

# Missing ZIP codes
invalid_zip = df[df['ZipCode'].isnull()]
print("Missing ZIP codes:", len(invalid_zip))

# Missing license dates
invalid_issue_dates = df[df['LicenseIssuedDate'].isnull()]
invalid_expire_dates = df[df['LicenseExpiredDate'].isnull()]
print("Missing issue dates:", len(invalid_issue_dates))
print("Missing expiration dates:", len(invalid_expire_dates))

```

Total rows: 721975

 NULL counts per column:

```

AnimalName 0
AnimalGender 0
AnimalBirthYear 0
BreedName 0
ZipCode 0
LicenseIssuedDate 0
LicenseExpiredDate 0
Extract Year 0
AnimalName_Clean 0
BreedName_Clean 0
BreedName_Final 0
breed_id 0
dtype: int64

```

 Invalid value checks:

```

Invalid birth years: 0
Missing ZIP codes: 0
Missing issue dates: 0
Missing expiration dates: 0

```

[56]: *# Birth Year Outlier Check (Before/After Verification)*

```

# Compute age
age = df['Extract Year'] - df['AnimalBirthYear']

# Identify outliers used previously
birth_outlier_mask = (
    (df['AnimalBirthYear'] < 1995) |
    (df['AnimalBirthYear'] > df['Extract Year']) |
    (age > 30)
)

# Count BEFORE cleaning (values may already be cleaned from earlier) print("Birth
year outliers BEFORE re-cleaning:", birth_outlier_mask.sum())

```

30

```
# Re-run the cleaning step (safe, idempotent — won't change good values)
```

```
df.loc[birth_outlier_mask, 'AnimalBirthYear'] = pd.NA
```

```
# Recompute age after cleaning
```

```
age_after = df['Extract Year'] - df['AnimalBirthYear']
```

```
# Check AGAIN using the same rules
```

```
remaining_outliers = (  
    (df['AnimalBirthYear'] < 1995) |  
    (df['AnimalBirthYear'] > df['Extract Year']) |  
    (age_after > 30)  
)
```

```
print("Birth year outliers AFTER cleaning:", remaining_outliers.sum())
```

```
# Show sample NA rows for transparency
```

```
df[df['AnimalBirthYear'].isna()].head()
```

Birth year outliers BEFORE re-cleaning: 0

Birth year outliers AFTER cleaning: 0

[56]: Empty DataFrame

Columns: [AnimalName, AnimalGender, AnimalBirthYear, BreedName, ZipCode, LicenseIssuedDate, LicenseExpiredDate, Extract Year, AnimalName_Clean, BreedName_Clean, BreedName_Final, breed_id]
Index: []

I ran a before/after check to verify that all implausible birth years were successfully corrected earlier in the pipeline based on biological impossibility rather than statistical distribution like Z scores. The results show that the number of outliers was already zero before the cleaning block was re-run and remained zero afterward. This confirms that all biologically unrealistic values were successfully removed during the earlier cleaning process, and the dataset now contains no age-related inconsistencies.

In contrast the column 'Extract Year' looked clean and reliable with values ranged from 2016 to 2024, and did not need any additional work.

The first transformation I applied was a full cleaning and normalization of the AnimalName field. The original dataset contained a wide range of inconsistent name formats, including upper case/lowercase variation, punctuation, special characters (e.g., "Ca\$h"), spacing issues, accented characters, and cases where names included stray symbols or formatting noise. These inconsistencies would create duplicate entries, hurt grouping and frequency analysis, and potentially cause issues during database insertion. To address this, I applied a systematic cleaning function that removed punctuation, normalized whitespace, stripped special characters, expanded initials when appropriate, and converted the names into a consistent title-case format. This transformation ensures that the AnimalName_Clean column is standardized, readable, and analytically reliable, preventing artificially inflated uniqueness and making downstream analysis

more accurate.

The second transformation focused on standardizing the BreedName field into a clean and

consistent 31

BreedName_Final column. The raw NYC dog licensing data included many different formatting issues. Several breed names appeared in reversed comma order such as “Schnauzer, Miniature,” “Poodle, Standard,” “Bull Dog, French,” and “Collie, Border,” so these needed to be reordered. The dataset also contained shortened breed words such as “Amer,” “Aus,” “Germ,” “Fr,” and “Eng,” which appeared inside names like “Amer Eskimo Dog” or “Aus Shepherd Mix.” These abbreviations had to be expanded to their full forms so the entries were clear and consistent. Breed mixes also appeared in many different styles, including “X,” “x,” slashes between breeds, and the word “cross.” In addition, many entries belonged to doodle or “poo” type breeds such as Goldendoodle, Labradoodle, Yorkie Poo, Cockapoo, Maltipoo, and Shih Poo. These needed to be cleaned and standardized so they appeared in a uniform format.

To resolve these issues, I used a multi-step cleaning process. I corrected comma-reversed names, expanded abbreviated terms, unified all mix indicators into a single consistent format, standardized doodle and poo type breed names, and organized multi-breed combinations in a clear alphabetical order. This approach ensured that different raw strings referring to the same breed were combined into one clean and reliable value. The final BreedName_Final column is now consistent and ready for analysis, and it can be safely used as a lookup table in PostgreSQL when assigning each breed a breed_id.

[57]: df.columns

```
[57]: Index(['AnimalName', 'AnimalGender', 'AnimalBirthYear', 'BreedName', 'ZipCode',  
          'LicenseIssuedDate', 'LicenseExpiredDate', 'Extract Year',  
          'AnimalName_Clean', 'BreedName_Clean', 'BreedName_Final', 'breed_id'],  
        dtype='object')
```

Below I will insert the data into the database in chunks with timing logs and fulfill the requirements for Part 2 of this final project as well as remove duplicates.

[58]: *# This was quite challenging getting the hang of this.*

```
# TIMER UTILITY  
# -----  
def log(msg):  
    print(f"[{time.strftime('%H:%M:%S')}] {msg}")  
  
# PART 1 — LOAD CSV  
t_start = time.time()  
log("Loading CSV...")  
  
t0 = time.time()  
df = pd.read_csv(
```

```

    "NYC_Dog_Licensing_Dataset_20251117.csv",
    dtype=str,
    encoding="utf-8",
    low_memory=False
)

```

32

```

log(f"CSV loaded in {time.time() - t0:.2f}s ({len(df):,} rows)")

```

PART 2 — CLEANING PIPELINE

```

log("Starting cleaning pipeline...")

```

```

df.rename(columns={"Extract Year": "Extract_Year"}, inplace=True)

```

```

df["animal_name"] = df["AnimalName"].astype(str).str.strip().str.title()

```

```

df["breed_name"] = (
    df["BreedName"]
    .astype(str).str.strip().str.title()
    .str.replace("/", " / ", regex=False)
    .str.replace(" ", " ")
    .str.replace("Mix", "Mixed", case=False)
    .str.strip()
)

```

```

gender_map = {
    "MALE": "M", "M": "M",
    "FEMALE": "F", "F": "F",
    "UNKNOWN": "U", "UNK": "U", "U": "U"
}
df["AnimalGender"] = (
    df["AnimalGender"].astype(str).str.upper().str.strip().map(gender_map).fillna("U")
)

```

```

df["AnimalBirthYear"] = pd.to_numeric(df["AnimalBirthYear"], errors="coerce")
df["Extract_Year"] = pd.to_numeric(df["Extract_Year"], errors="coerce")

```

```

df["LicenseIssuedDate"] = pd.to_datetime(df["LicenseIssuedDate"],
    errors="coerce").dt.strftime("%Y-%m-%d")
df["LicenseExpiredDate"] = pd.to_datetime(df["LicenseExpiredDate"],
    errors="coerce").dt.strftime("%Y-%m-%d")

```

```

df["ZipCode"] = (
    df["ZipCode"].astype(str)
    .str.replace(".0", "", regex=False)

```

```

        .str.replace(r"\D", "", regex=True)
        .str.zfill(5)
    )

```

Keep only NYC ZIP codes

33

```

log("Filtering to NYC ZIP codes...")

df["ZipCode"] = pd.to_numeric(df["ZipCode"], errors="coerce") df =
df[df["ZipCode"].between(10000, 14999)]

df["ZipCode"] = (
    df["ZipCode"].astype(int).astype(str).str.zfill(5)
)

log(f"Remaining rows after NYC ZIP filter: {len(df):,}")

```

PART 3 — SQL-FRIENDLY DF

```

df_sql = pd.DataFrame({
    "animal_name": df["animal_name"],
    "animal_gender": df["AnimalGender"],
    "animal_birth_year": df["AnimalBirthYear"],
    "license_issued_date": df["LicenseIssuedDate"],
    "license_expired_date": df["LicenseExpiredDate"],
    "extract_year": df["Extract_Year"],
})

```

PART 4 — CONNECT TO PostGRE

```

log("Connecting to PostgreSQL...")

conn = psycopg2.connect(
    host="localhost",
    dbname="nyc_dog_licensing",
    user="postgres",
    password="Python"
)

cur = conn.cursor()

```

PART 5 — LOAD LOOKUP TABLES

```
log("Loading lookup tables...")
```

```
unique_breeds = df["breed_name"].unique()
cur.executemany(
    "INSERT INTO breeds (breed_name) VALUES (%s) ON CONFLICT DO
    NOTHING;", [(b,) for b in unique_breeds]
)
```

34

```
conn.commit()
```

```
unique_zips = df["ZipCode"].unique()
cur.executemany(
    "INSERT INTO zipcodes (zip_code) VALUES (%s) ON CONFLICT DO
    NOTHING;", [(z,) for z in unique_zips]
)
conn.commit()
```

PART 6 — FOREIGN KEY LOOKUP

```
log("Mapping foreign keys...")
```

```
cur.execute("SELECT breed_id, breed_name FROM breeds;")
breed_lookup = {name: bid for bid, name in cur.fetchall()}
```

```
cur.execute("SELECT zip_id, zip_code FROM zipcodes;")
zip_lookup = {zc: zid for zid, zc in cur.fetchall()}
```

```
df_sql["breed_id"] = df["breed_name"].map(breed_lookup)
df_sql["zip_id"] = df["ZipCode"].map(zip_lookup)
```

```
# PART 7 — CLEAN INVALID YEARS AND REMOVE DUPLICATES
FOR SQL print("\n=== FIXING INVALID BIRTH YEARS (1995–NOW)
===") current_year = pd.Timestamp.now().year
```

```
df_sql["animal_birth_year"] = pd.to_numeric(df_sql["animal_birth_year"], _
    errors="coerce")
```

```
df_sql.loc[
    (df_sql["animal_birth_year"] < 1995) |
```

```

        (df_sql["animal_birth_year"] > current_year),
        "animal_birth_year"
    ] = None

print("\n=== REMOVING DUPLICATE DOG RECORDS ===")

df_sql = df_sql.drop_duplicates(
    subset=[
        "animal_name",
        "animal_gender",
        "animal_birth_year",
        "license_issued_date",
        "breed_id",

        "zip_id"
    ],
    keep="first"
).reset_index(drop=True)

print("Remaining rows:", len(df_sql))

```

35

```

# PART 8 — FINAL CLEANUP (REMOVE NA / NAN)
log("Finalizing numeric conversions...")

df_sql = df_sql.where(pd.notnull(df_sql), None)
df_sql = df_sql.astype(object)

df_sql = df_sql.applymap(lambda x: None if pd.isna(x) else x)

# PART 9 — CONVERT TO PURE PYTHON ROWS
log("Converting DataFrame to Python rows...")

rows = []
for _, r in df_sql.iterrows():

    # safe function to convert numeric fields
    def safe_int(val):
        return None if val is None or pd.isna(val) else int(val)

    rows.append((
        r["animal_name"],
        r["animal_gender"],

```

```

        safe_int(r["animal_birth_year"]),
        r["license_issued_date"],
        r["license_expired_date"],
        safe_int(r["extract_year"]),
        safe_int(r["breed_id"]),
        safe_int(r["zip_id"])
    ))

```

```
log(f"Converted {len(rows):,} rows to pure Python tuples.")
```

PART 10 — CHUNKED INSERTS (:

```
log("Starting inserts...")
```

36

```

insert_query = """
INSERT INTO dogs (
    animal_name,
    animal_gender,
    animal_birth_year,
    license_issued_date,
    license_expired_date,
    extract_year,
    breed_id,
    zip_id
) VALUES (%s,%s,%s,%s,%s,%s,%s,%s)
ON CONFLICT ON CONSTRAINT unique_dog_record DO
NOTHING; """

```

```
chunk_size = 100_000
```

```
total_rows = 0
```

```

for i in range(0, len(rows), chunk_size):
    chunk = rows[i:i+chunk_size]
    cur.executemany(insert_query, chunk)
    conn.commit()

```

```
total_rows += len(chunk)
```

```
log(f"Inserted {total_rows:,} rows so far...")
```

```
cur.close()
```

```
conn.close()
```

```
log(f"Inserted {total_rows:,} total rows.") log(f"Total ETL time:
```



```
{time.time() - t_start:.2f}s")
```

```
[00:49:28] Loading CSV...
[00:49:29] CSV loaded in 0.69s (722,864 rows)
[00:49:29] Starting cleaning pipeline...
[00:49:33] Filtering to NYC ZIP codes...
[00:49:33] Remaining rows after NYC ZIP filter: 722,211
[00:49:33] Connecting to PostgreSQL...
[00:49:33] Loading lookup tables...
[00:49:34] Mapping foreign keys...
```

=== FIXING INVALID BIRTH YEARS (1995–NOW) ===

=== REMOVING DUPLICATE DOG RECORDS ===

```
Remaining rows: 614980
[00:49:34] Finalizing numeric conversions...
```

37

```
C:\Users\adamk\AppData\Local\Temp\ipykernel_20656\2738510142.py:173:
FutureWarning: DataFrame.applymap has been deprecated. Use DataFrame.map
instead.
```

```
df_sql = df_sql.applymap(lambda x: None if pd.isna(x) else x)
```

```
[00:49:36] Converting DataFrame to Python rows...
[00:49:56] Converted 614,980 rows to pure Python tuples.
[00:49:56] Starting inserts...
[00:50:11] Inserted 100,000 rows so far...
[00:50:25] Inserted 200,000 rows so far...
[00:50:38] Inserted 300,000 rows so far...
[00:50:51] Inserted 400,000 rows so far...
[00:51:03] Inserted 500,000 rows so far...
[00:51:18] Inserted 600,000 rows so far...
[00:51:20] Inserted 614,980 rows so far...
[00:51:20] Inserted 614,980 total rows.
[00:51:20] Total ETL time: 111.61s
```

[59]:

```
print("\n=====")
      print(" DATA-VOLUME SANITY CHECK (POST ETL)")
print("=====\\n")

# 1. Final row count in df_sql
print(f"Rows prepared for DB insert: {len(df_sql):,}")

# 2. Check NULLs in DF that is actually inserted
print("\nNULL counts (final df_sql):")
print(df_sql.isna().sum())
print("-" * 40)
```

3. Count removed invalid birth years

```
current_year = pd.Timestamp.now().year
invalid_birth_years = df_sql[
    (df_sql['animal_birth_year'].notna()) &
    ((df_sql['animal_birth_year'] < 1995) |
     (df_sql['animal_birth_year'] > current_year))
]

print(f"Invalid (removed) birth years: {len(invalid_birth_years):,}")
```

4. Missing ZIP IDs after mapping

```
missing_zip_id = df_sql['zip_id'].isna().sum()
print(f"Missing ZIP foreign keys: {missing_zip_id:,}")
```

5. Missing Breed IDs after mapping

```
missing_breed_id = df_sql['breed_id'].isna().sum()
print(f"Missing Breed foreign keys: {missing_breed_id:,}") 38
```

6. Missing required issue dates

```
missing_issue = df_sql['license_issued_date'].isna().sum()
print(f"Missing license_issued_date: {missing_issue:,}")
```

7. Duplicate check AFTER SQL dedupe subset

```
dedupe_cols_sql = [
    "animal_name",
    "animal_gender",
    "animal_birth_year",
    "license_issued_date",
    "breed_id",
    "zip_id"
]

dupes_after = df_sql[df_sql.duplicated(subset=dedupe_cols_sql, keep=False)]
print(f"Remaining duplicates in df_sql: {len(dupes_after):,}")
```

8. Null birth years (allowed but counted)

```
null_birth_years = df_sql['animal_birth_year'].isna().sum()
print(f"Records with NULL birth year: {null_birth_years:,}")
```

9. Expired date < issued date check

```
invalid_date_order = df_sql[
    df_sql['license_expired_date'].notna() &
    (df_sql['license_expired_date'] < df_sql['license_issued_date']) ]
print(f"Records with expiration < issued date: {len(invalid_date_order):,}")
```

```
print("\n=====
==") print(" END SANITY CHECK — SUMMARY ABOVE")
print("=====\\n")
```

```
=====
DATA-VOLUME SANITY CHECK (POST ETL)
=====
```

Rows prepared for DB insert: 614,980

NULL counts (final df_sql):

```
animal_name 0
animal_gender 0
animal_birth_year 130
license_issued_date 0
license_expired_date 49
extract_year 0
breed_id 0
zip_id 0
```

39

dtype: int64

```
-----
Invalid (removed) birth years: 0
Missing ZIP foreign keys: 0
Missing Breed foreign keys: 0
Missing license_issued_date: 0
Remaining duplicates in df_sql: 0
Records with NULL birth year: 130
Records with expiration < issued date: 0
```

```
=====
END SANITY CHECK — SUMMARY ABOVE
=====
```

Part 2A:

For the ETL process, I built a full, step-by-step pipeline in Python that loads the raw NYC Dog Licensing CSV file, cleans each field, converts everything into the correct data types, and finally inserts the data into my PostgreSQL database in optimized batches. I used pandas to handle all of the data wrangling: renaming columns, fixing inconsistent capitalization, normalizing breed names, standardizing gender codes, validating numeric fields like birth years, and cleaning all dates so that they follow valid chronological rules. After the cleaning phase, I created a SQL-ready DataFrame and connected to PostgreSQL using psycopg2. Lookup tables for breeds and ZIP codes were loaded first, followed by a foreign-key mapping stage to match each dog to its breed_id and zip_id. For performance, I inserted the dog records using chunks to execute

many batches, which let me efficiently load 614K+ rows while logging the insert times. This ETL pipeline guarantees that only validated, clean, constraint-safe data enters the database. Also, in my schema I have a CONSTRAINT on 'unique_dog_record', where if a row matches an existing row on: 'animal_name,' 'animal_gender,' 'animal_birth_year,' 'license_issued_date,' 'breed_id,' and 'zip_id' this tells PostgreSQL to block it. Thus, I realized I had a lot of duplicates approximately a little over 100,000. Thus, duplicates were removed above.

Part 2B:

While cleaning the dataset, I ran into a bunch of messy fields that needed to be fixed before anything could go into the database. For names, I standardized the animal_name and breed_name columns by stripping extra spaces, fixing capitalization, and cleaning up weird formatting. I also made sure breed names were consistent by replacing things like "Mix" with "Mixed" and formatting breeds that had slashes or multiple words.

Dates needed even more attention. There were cases where the expiration date was missing or was earlier than the issued date. To fix this, I converted all date fields to real datetime objects, filled missing expiration dates with the issued date, and corrected any reversed dates so that they always satisfy the rule that the issued date must be earlier than or equal to the expiration date. This keeps everything in a valid chronological order and satisfies the constraint in my dogs table.

The ZIP codes in the raw dataset were also all over the place. I cleaned them by stripping out anything that wasn't a number, removing decimals, zero-padding everything to five digits, and fixing inconsistent formats. One record had a ZIP code from Alaska, which isn't part of NYC, so I removed that entry entirely and set the zipcode to filter only New York.

40

```
[60]: print(df.columns.tolist())
```

```
['AnimalName', 'AnimalGender', 'AnimalBirthYear', 'BreedName', 'ZipCode',  
'LicenseIssuedDate', 'LicenseExpiredDate', 'Extract_Year', 'animal_name',  
'breed_name']
```

```
[61]: # Drop duplicates also on my DF
```

```
dedupe_cols = [  
    "animal_name", # cleaned name  
    "AnimalGender", # original column  
    "AnimalBirthYear", # original column  
    "LicenseIssuedDate", # original column  
    "breed_name", # cleaned name  
    "ZipCode" # original column  
]
```

```
df_deduped = df.drop_duplicates(subset=dedupe_cols, keep="first")
```

```
df = df_deduped
```

Part 2 is complete on this notebook of 'Data Ingestion & Preparation, and I will now focus on Part 4 below:

- A. Exploratory notebook (20 pts) • Use pandas-SQLAlchemy connection to pull data. •
- At least three visualizations (one must be interactive or geospatial if data allow). •
- Narrative explaining trends, anomalies, and limitations.

```
[62]: from sqlalchemy import create_engine
```

```
[63]: # Create SQLAlchemy Engine
```

```
engine = create_engine("postgresql+psycopg2://postgres:Python@localhost/
    ↳nyc_dog_licensing")
```

```
[64]: # Test the connection
```

```
pd.read_sql("SELECT COUNT(*) FROM dogs;", engine)
```

```
[64]: count
```

```
0 615833
```

```
[65]: # Pull some small tables in
```

```
breeds_df = pd.read_sql("SELECT * FROM breeds;", engine)
```

```
zipcodes_df = pd.read_sql("SELECT * FROM zipcodes;", engine)
```

```
[66]: # Verify we're getting only NYC Zipcodes
```

```
pd.read_sql("""
    SELECT
        MIN(zip_code) AS min_zip_code,
        MAX(zip_code) AS max_zip_code
```

41

```
FROM zipcodes;
""", engine)
```

```
[66]: min_zip_code max_zip_code
```

```
0 10001 14905
```

```
[67]: # Pull large table (dogs) using SQL queries
```

```
dogs_sample = pd.read_sql("SELECT * FROM dogs LIMIT 50;", engine)
dogs_sample.head()
```

```
[67]: dog_id animal_name animal_gender animal_birth_year license_issued_date \ 0 1 Paige F
2014 2014-09-12 1 2 Yogi M 2010 2014-09-12 2 3 Ali M 2014 2014-09-12 3 4 Queen F 2013
2014-09-12 4 5 Lola F 2009 2014-09-12
```

```
license_expired_date extract_year breed_id zip_id \
0 2017-09-12 2016 1 1
1 2017-10-02 2016 2 2
2 2019-09-12 2016 3 3
3 2017-09-12 2016 4 3
4 2017-10-09 2016 5 4
```

```
created_at
0 2025-12-06 22:50:55.357488
1 2025-12-06 22:50:55.357488
2 2025-12-06 22:50:55.357488
3 2025-12-06 22:50:55.357488
4 2025-12-06 22:50:55.357488
```

[68]: # Check total licensed dogs per year

```
dogs_by_year = pd.read_sql("""
    SELECT extract_year, COUNT(*) AS total
    FROM dogs
    GROUP BY extract_year
    ORDER BY extract_year;
""", engine)
dogs_by_year
```

[68]: extract_year total

```
0 2016 112614
1 2017 73592
2 2018 70402
3 2022 210943
4 2023 89800
5 2024 58482
```

42

The first thing that stands out is that 2016 has a very high count (112,576), then the numbers drop steadily through 2017 and 2018. That doesn't necessarily mean there were fewer dogs in NYC; it likely reflects differences in how many owners renewed or how consistently the city processed licenses in those years. NYC licensing is voluntary and not everyone renews on time, so you naturally see fluctuations that don't match actual ownership trends.

The most dramatic spike is in 2022, with over 211,000 licenses. This is almost double the count from 2016 and way above every other year in the dataset. This aligns with a COVID-era effect: a lot of people adopted pets during and after the pandemic, and NYC also pushed more online licensing and renewal reminders. The dataset includes that surge very clearly.

After 2022, the counts drop again for 2023 and 2024. A decline after a record-breaking spike is expected because the 2022 number represents both new adoptions and delayed renewals from the pandemic years. Once that backlog clears, the totals settle closer to normal levels. The 2024 number is lower partly because the dataset only contains whatever was recorded up to the extraction date and may not include the full year.

One thing to note is that some years (like 2019–2021) are missing entirely. That doesn't indicate zero dogs; it just means those extract years weren't included in the CSV you were given. The NYC open data portal sometimes skips years in certain releases, depending on how the dataset was exported.

Overall, the year-to-year counts likely reflect administrative cycles, renewal behavior, and COVID

era changes rather than real changes in how many dogs live in NYC. The big spike in 2022 is the clearest trend, and the dips around it are tied to how the city processes its data, not the actual dog population.

When I looked at the number of dog licenses issued per year, the dataset shows several clear jumps and dips that are worth explaining.

[69]: *# Pull top 20 breeds directly from PostgreSQL*

```
top_breeds = pd.read_sql("""
    SELECT
        b.breed_name,
        COUNT(*) AS count
    FROM dogs d
    JOIN breeds b
        ON d.breed_id = b.breed_id
    GROUP BY b.breed_name
    ORDER BY count DESC
    LIMIT 20;
""", engine)

print(top_breeds)

# Remove "Unknown" to improve clarity
clean_breeds = top_breeds[top_breeds["breed_name"] != "Unknown"]
```

43

```
# Sort ascending for a clean horizontal bar plot
clean_breeds = clean_breeds.sort_values("count", ascending=True)

# horizontal bar chart
plt.figure(figsize=(12, 10))
plt.barh(clean_breeds["breed_name"], clean_breeds["count"])

plt.title("Top 19 Known Dog Breeds in NYC (Unknown Removed)", fontsize=18, _
pad=15)
plt.xlabel("Number of Licensed Dogs", fontsize=14)
plt.ylabel("Breed", fontsize=14)

plt.grid(axis="x", linestyle="--", linewidth=0.5)
plt.xticks(fontsize=12)
plt.yticks(fontsize=12)

plt.tight_layout()
plt.show()
```

breed_name count

0 Unknown 59400
1 Yorkshire Terrier 34936
2 Shih Tzu 32297
3 Chihuahua 24481
4 Labrador Retriever 18463
5 Maltese 18115
6 Labrador Retriever Crossbreed 16138
7 American Pit Bull Mixed / Pit Bull Mixed 14737
8 Shih Tzu Crossbreed 11991
9 Pomeranian 11828
10 American Pit Bull Terrier / Pit Bull 11387
11 Poodle Crossbreed 10791
12 Havanese 10551
13 Chihuahua Crossbreed 9523
14 Golden Retriever 9219
15 French Bulldog 8830
16 German Shepherd Dog 8698
17 Beagle 8669
18 Goldendoodle 7818
19 Poodle 7594



When I looked at the most common breeds in the database, the first thing that jumps out is that 'Unknown' shows up as the #1 category. That's because a lot of messy or incomplete breed entries in the raw data got standardized into 'Unknown' during cleaning, and the dataset originally had some 'Unknown' values as well. So this category is inflated and shouldn't be interpreted as an actual breed.

Outside of 'Unknown,' the results make a lot of sense for NYC. The top dogs are mostly small breeds like Yorkshire Terriers, Shih Tzus, Chihuahuas, Maltese, and Pomeranians. These breeds are extremely popular in dense urban environments because they're easier to manage in apartments.

Mixed breeds also show up heavily such as Labrador Retriever Crossbreed, Shih Tzu Crossbreed, Poodle Crossbreed, and various Pit Bull mixes where from my personal knowledge most rescue and shelter dogs come from. Thus, this is a reflection of how likely adoption and rescue dogs are in NYC.

Larger purebreds like Labrador Retrievers, Golden Retrievers, German Shepherds, and French Bulldogs are still well represented, just not at the same scale as the small breeds.

[70]: *# SQL query to count male vs female dogs and unknown*
gender_counts = pd.read_sql("""

SELECT

```
        animal_gender,  
        COUNT(*) AS total  
    FROM dogs  
    GROUP BY animal_gender  
    ORDER BY animal_gender;  
""", engine)
```

```
gender_counts
```

```
# Sort categories manually for clean ordering
```

```
gender_counts = gender_counts.sort_values("animal_gender")
```

```
plt.figure(figsize=(8, 5))
```

```
plt.barh(gender_counts["animal_gender"], gender_counts["total"])
```

```
plt.title("Distribution of Dog Genders in NYC", fontsize=18, pad=15)
```

```
plt.xlabel("Number of Dogs", fontsize=14)
```

```
plt.ylabel("Gender", fontsize=14)
```

```
plt.grid(axis="x", linestyle="--", linewidth=0.5)
```

```
plt.xticks(fontsize=12)
```

```
plt.yticks(fontsize=12)
```

```
plt.tight_layout()
```

```
plt.show()
```



When I pulled the gender counts from the database, the numbers were pretty much what I expected. Male dogs show up slightly more than females, but they're both close in volume. The "U" category is small and mostly comes from messy values in the raw dataset that got standardized during cleaning. It doesn't mean the data was missing, just that the original entries weren't formatted cleanly. Overall, the gender distribution looks normal and lines up with what you'd expect in a dataset this size.

[71]: # *Dogs per ZIP code*

```
zip_counts = pd.read_sql("""
    SELECT z.zip_code, COUNT(*) AS total
    FROM dogs d
    JOIN zipcodes z ON d.zip_id = z.zip_id
    GROUP BY z.zip_code
    ORDER BY total DESC;
""", engine)
zip_counts
```

```
[71]: zip_code total
0 10025 13757
1 10023 11132
2 11201 10778
3 11215 10669
4 10024 10494
... ..
638 11313 1
```

```
639 11703 1
```

```
640 11301 1
641 11298 1
642 11287 1
```

```
[643 rows x 2 columns]
```

With the updated cleaning pipeline, I'm now only loading valid New York ZIP codes into the database. The out-of-state values I was seeing earlier (like Alaska and other regions) are completely gone, which confirms the filter is working correctly.

Altair works a lot more smoothly when the data is already in a dataframe, so I'm going to keep using the dataframe I loaded earlier instead of repeatedly querying SQL for every chart. At the start of this project I did my EDA directly in pandas anyway, so it makes sense to finish the rest of this exploratory notebook in the same workflow.

Date Handling and Time-Based Analysis:

ff Convert the date columns to datetime format

```
[72]: df['LicenseIssuedDate'] = pd.to_datetime(df['LicenseIssuedDate'],   
      _errors='coerce')
      df['LicenseExpiredDate'] = pd.to_datetime(df['LicenseExpiredDate'],   
      _errors='coerce')
```

```
[73]: df['Issued_Year'] = df['LicenseIssuedDate'].dt.year
      df['Issued_Month'] = df['LicenseIssuedDate'].dt.month
      df['Expire_Year'] = df['LicenseExpiredDate'].dt.year
      df['Expire_Month'] = df['LicenseExpiredDate'].dt.month
      df['Issued_Week'] = df['LicenseIssuedDate'].dt.isocalendar().week
      df['Expire_Week'] = df['LicenseExpiredDate'].dt.isocalendar().week
```

```
[74]: # Clean
      today = pd.Timestamp.today()
      df['Is_Expired'] = df['LicenseExpiredDate'] < today
```

```
[75]: # The dataset was last updated in 2022, so any birth year after 2022 is   
      df = df[(df['AnimalBirthYear'] >= 2000) & (df['AnimalBirthYear'] <= 2022)]
```

```
[76]: # Ensure birth year is numeric (handles strings like '2014.0') df['AnimalBirthYear'] =   
      pd.to_numeric(df['AnimalBirthYear'], errors='coerce')
```

```
# Get current year
current_year = pd.Timestamp.today().year
```

```
# Calculate age today
df['Dog_Age_Today'] = current_year - df['AnimalBirthYear']
```

C:\Users\adamk\AppData\Local\Temp\ipykernel_20656\1097889868.py:2:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame. Try using
.loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy df['AnimalBirthYear'] =
pd.to_numeric(df['AnimalBirthYear'], errors='coerce')
C:\Users\adamk\AppData\Local\Temp\ipykernel_20656\1097889868.py:8:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame. Try using
.loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
df['Dog_Age_Today'] = current_year - df['AnimalBirthYear']

Suspiciousness check:

ff Are there any suspicious or incorrect values in the License_Duration_Days column (such as negative durations or unusually long license periods)? If so, how would these be identified and handled?

ff Also, calculate and add a new column "Licence_Duration_Days" showing how many days each license lasted.

[77]: *# Calculate the total number of days each dog license was valid # by
subtracting the issue date from the expiration date.*

```
df['License_Duration_Days'] = (df['LicenseExpiredDate'] -  
~df['LicenseIssuedDate']).dt.days
```

C:\Users\adamk\AppData\Local\Temp\ipykernel_20656\880292395.py:3:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame. Try using
.loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
df['License_Duration_Days'] = (df['LicenseExpiredDate'] -
df['LicenseIssuedDate']).dt.days

[78]: *# Check for negative values that occur when the expiration
date is earlier than the issue date, which is impossible for a valid license.*

```
df[df['License_Duration_Days'] < 0][['LicenseIssuedDate', 'LicenseExpiredDate',  
~'License_Duration_Days']]
```

[78]: Empty DataFrame

Columns: [LicenseIssuedDate, LicenseExpiredDate, License_Duration_Days] 49

Index: []

[79]: # Negative durations (expiration before issuance)

```
negative_durations = df[df["License_Duration_Days"] < 0]
```

Zero-day durations (issued and expired same day)

```
zero_durations = df[df["License_Duration_Days"] == 0]
```

```
short_durations = df[df["License_Duration_Days"] < 30]
```

Very long durations (>5 years = 1825 days)

```
long_durations = df[df["License_Duration_Days"] > 1825]
```

Print summary

```
print("Suspicious License Duration values:")
```

```
print(f"Negative durations: {len(negative_durations)}")
```

```
print(f"Zero-day durations : {len(zero_durations)}")
```

```
print(f"Short durations (<30 days): {len(short_durations)}")
```

```
print(f"Very long durations (>5 years): {len(long_durations)}")
```

Suspicious License Duration values:

Negative durations: 0

Zero-day durations : 0

Short durations (<30 days): 951

Very long durations (>5 years): 20983

[80]: # Flag less than 30 days, and over 5 years

```
df['Suspiciously_Short_Duration'] = df['License_Duration_Days'] < 30
```

```
df['Suspiciously_Long_Duration'] = df['License_Duration_Days'] > 1825
```

C:\Users\adamk\AppData\Local\Temp\ipykernel_20656\1820734867.py:2:

SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame. Try using

.loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: <https://pandas.pydata.org/pandas>

docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df['Suspiciously_Short_Duration'] = df['License_Duration_Days'] < 30
```

C:\Users\adamk\AppData\Local\Temp\ipykernel_20656\1820734867.py:3:

SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame. Try using

.loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: <https://pandas.pydata.org/pandas>

docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df['Suspiciously_Long_Duration'] = df['License_Duration_Days'] > 1825
```

According to NYC.gov, all dogs in New York City must be licensed, and owners can purchase a license that is valid for one year or up to five years. Because of these fixed validity periods, I

examined the 'License_Duration_Days' column to identify any values that fall outside this expected range.

During this check, I found 986 entries with durations under 30 days. Since NYC dog licenses should be valid for at least one full year, any duration shorter than one month is unusual and should be treated as suspicious. These short durations most likely reflect data-entry mistakes, placeholder expiration dates, the dog's death shortly after licensing, or incomplete renewal records where a temporary expiration date was entered before the full renewal was finalized. These cases should be flagged or excluded from duration-based analyses because they do not represent valid license periods. I chose to flag them for now.

I also found a significant number of licenses with durations that exceed the maximum 5-year validity period allowed by NYC law. This is likely due to how the Dog Licensing System records renewals: instead of creating a new entry for each renewal, the system appears to overwrite the expiration date in the original record. As a result, multiple renewal cycles accumulate into unusually long license durations. This behavior explains the 43,181 records with durations longer than five years and should be interpreted as administrative system behavior, not fraudulent activity. I kept these values in the dataset but flagged them for awareness and potential exclusion when analyzing license duration specifically.

- Create a visualization showing the average license duration by dog breed. Which breed typically has the longest license duration?

[81]: *# Calculate the total number of days each dog license was valid # by subtracting the issue date from the expiration date.*

```
df['License_Duration_Days'] = (df['LicenseExpiredDate'] - _
    - df['LicenseIssuedDate']).dt.days
```

C:\Users\adamk\AppData\Local\Temp\ipykernel_20656\880292395.py:3:

SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame. Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
 df['License_Duration_Days'] = (df['LicenseExpiredDate'] -
 df['LicenseIssuedDate']).dt.days

[82]: *# Calculate the average license duration by breed*

```
breed_duration = (
    df.groupby('BreedName', as_index=False)['License_Duration_Days']
        .mean()
        .sort_values('License_Duration_Days', ascending=False)
)
```

```
top20_breeds = breed_duration.head(20)
```

[83]: chart = (
 alt.Chart(top20_breeds)

51

```

.mark_bar(color="#4C72B0") # ← blue tone
.encode(
    x=alt.X('License_Duration_Days:Q', title='Average License Duration _
~(Days)'),
    y=alt.Y('BreedName:N', sort='-x', title='Dog Breed'),
    tooltip=[
        alt.Tooltip('BreedName:N', title='Breed'),
        alt.Tooltip('License_Duration_Days:Q', format=',.0f', title='Avg _
~Duration (Days)')
    ]
)
.properties(
    title='Top 20 Dog Breeds by Average License Duration',
    width=650,
    height=450
)
)
chart

```

[83]: alt.Chart(...)

This bar chart ranks the top 20 dog breeds by their average license duration, showing which breeds tend to maintain the longest continuous licensing periods in New York City. The Labrador–Pointer Mixed stands out as a clear outlier, with an average duration exceeding 2,300 days (roughly 6.3 years). The Red Poodle follows closely behind with a similarly extended duration.

Below these top two breeds, most of the remaining groups from Spaniel/Chihuahua mixes down through the German shepherd/Corgi mix—cluster around the 1,600–1,900 day range. This suggests that many owners of these breeds consistently renew their licenses across multiple cycles, leading to durations longer than the standard 5-year limit. As discussed earlier, this pattern is likely due to the NYC Dog Licensing System updating the expiration date upon renewal rather than generating new rows, causing renewals to accumulate in a single record.

Toward the bottom of the top 20 list, the average duration dips slightly, with breeds like the Miniature Poodle/Cocker Spaniel averaging closer to 1,300–1,400 days (about 3.4–3.5 years). Overall, the chart illustrates which breeds tend to have the longest continuous license histories and reflects both owner behavior and underlying administrative system structure.

ff Create a new column called “Dog_Age_Today” that categorizes dogs into the following groups: “Puppy” (0-1), “Teenager” (1-3), “Adult” (4-7), and “Old” (8+).

```

[84]: # Define age-grouping function
def age_group(age):
    if pd.isna(age):
        return None # Missing age
    elif age <= 1:

```



```
    return 'Puppy' # 0–1 years  
elif age <= 3:  
    return 'Teenager' # 1–3 years
```